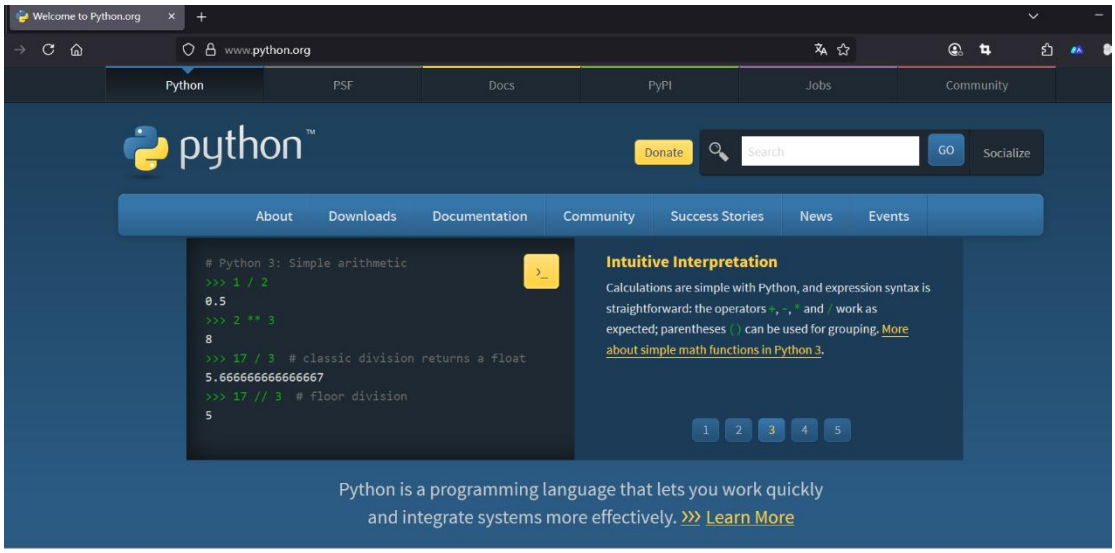
开发环境

1. python 安装

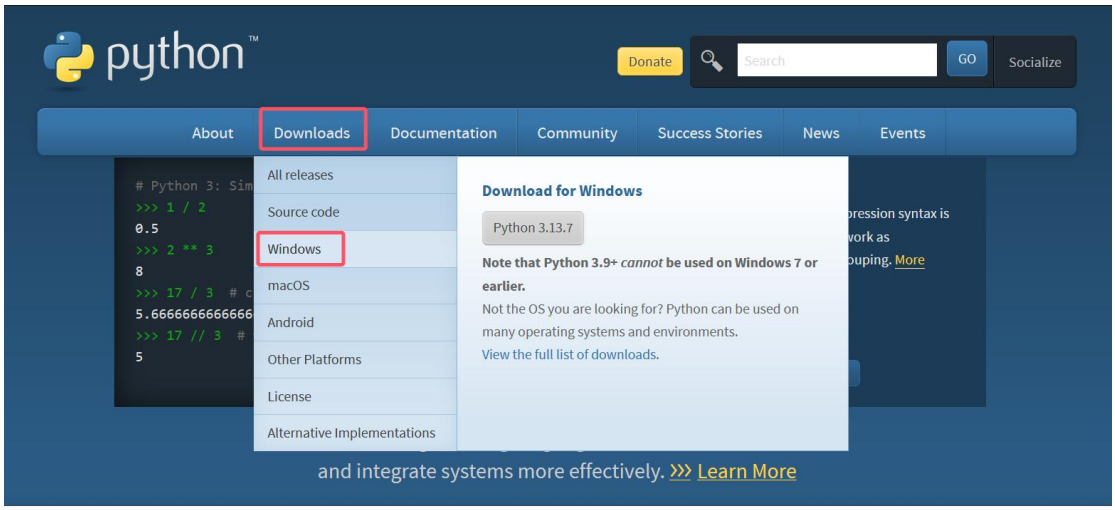
单独安装 python 这步可跳过，如果熟悉 conda 的使用的话。

python 下载

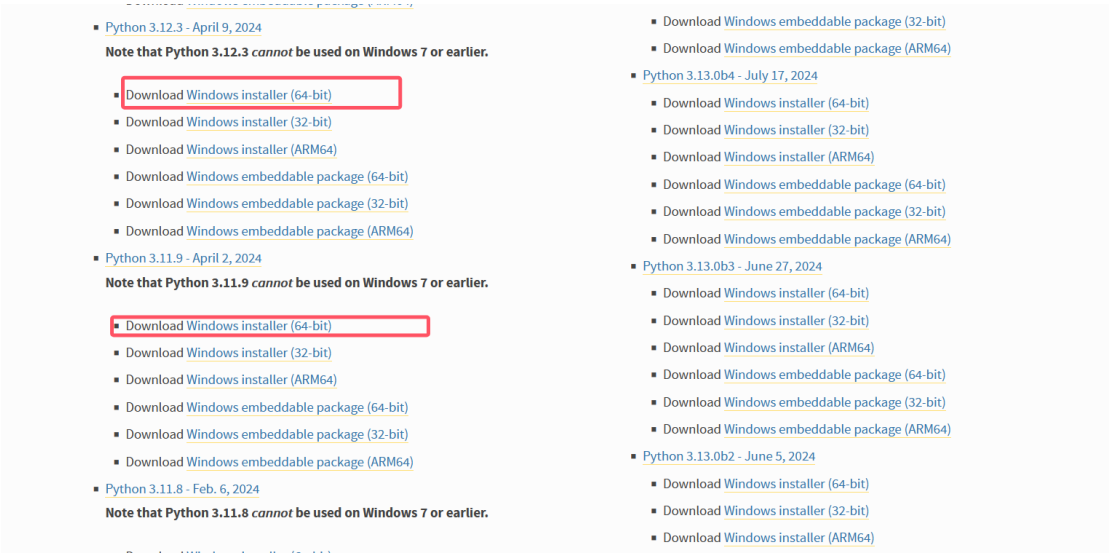
进入 [python 官网]



选择合适的系统版本，这里以 windows 系统为例。

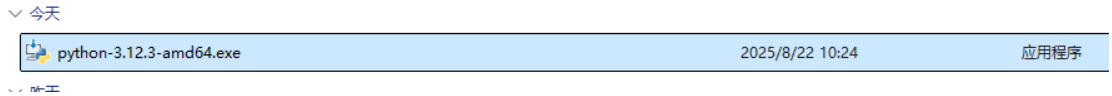


选择 python 版本，3.9<=版本<最新版。

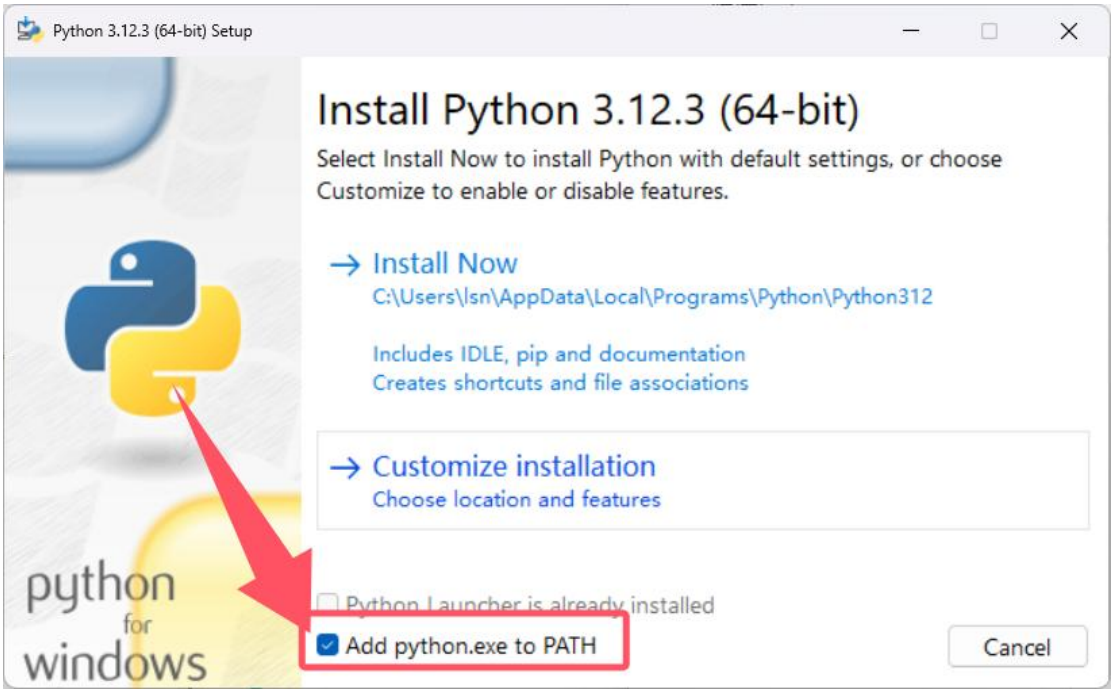


python 安装

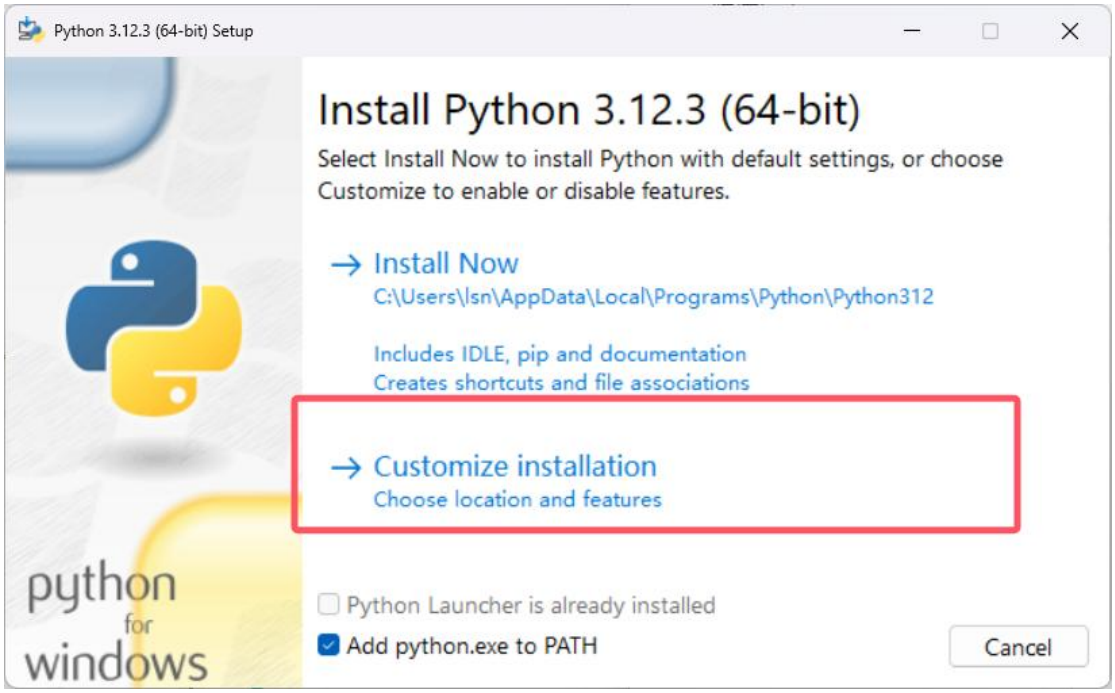
下载 installer 之后，点击安装。



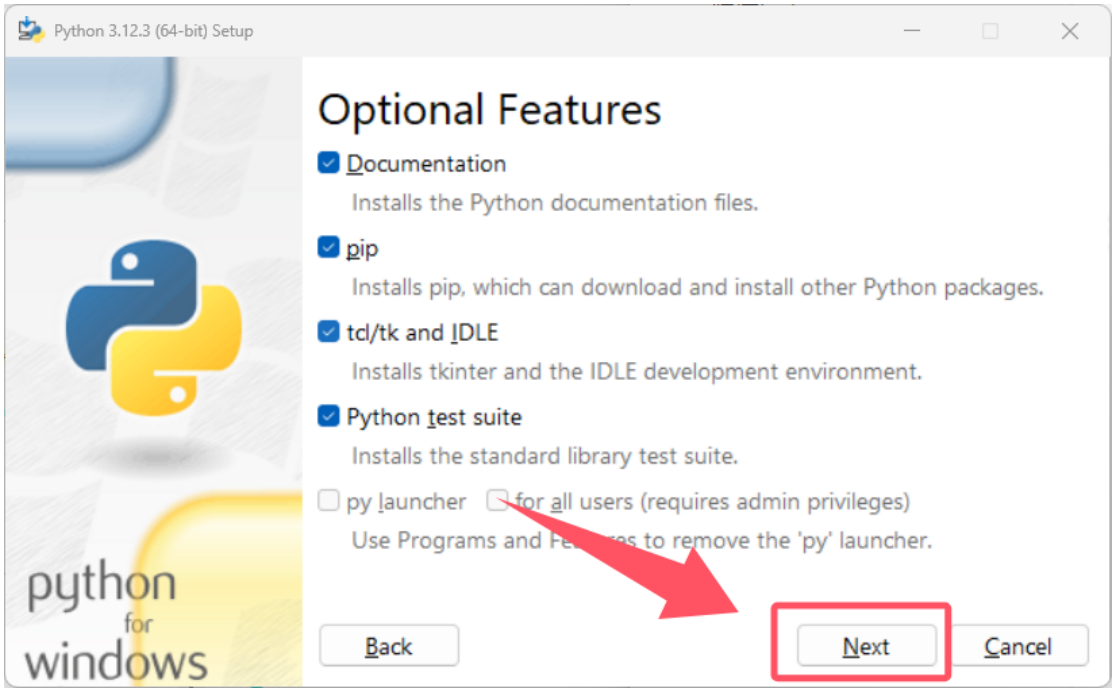
如果没装过 python，可以点击勾选下方的 Add python.exe tp PATH。



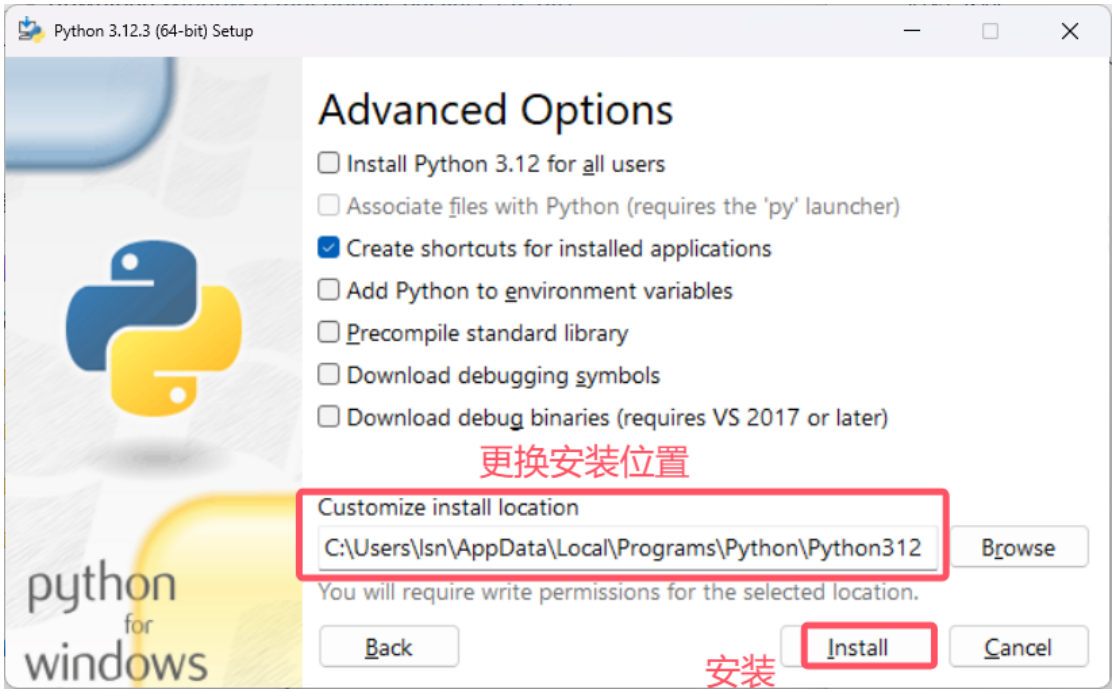
然后点击自定义安装。



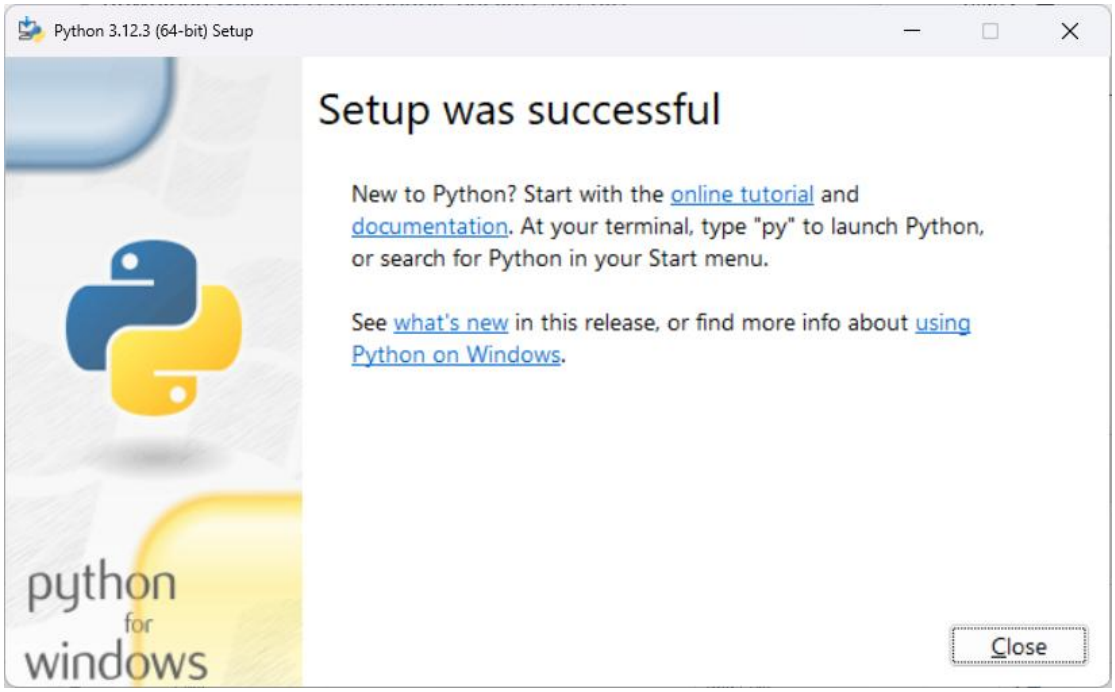
点击 Next。



这里选项保持默认即可，可更换下方 python 的安装位置。点击 Install 安装。

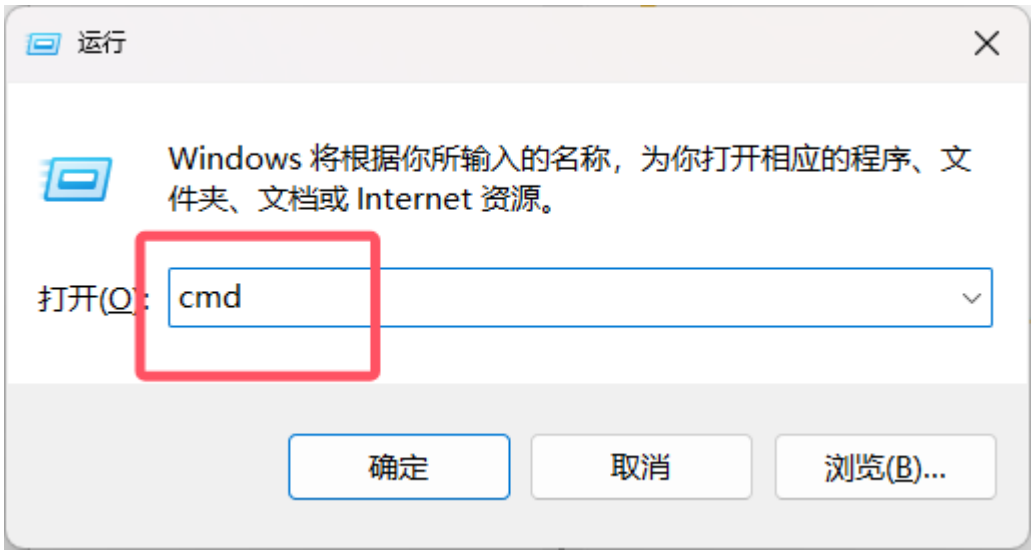


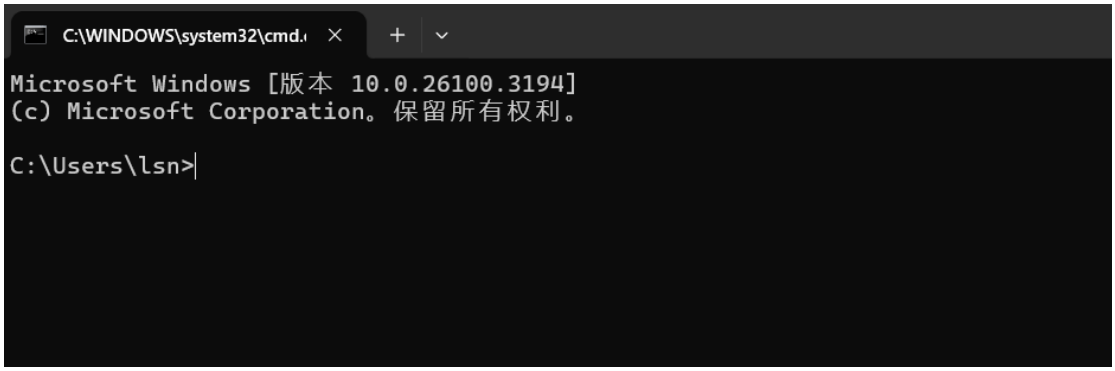
安装完成之后，关闭窗口。



python 验证

安装完成之后，打开命令行终端（以 window 为例），按下键盘 WIN+R，屏幕左下方会显示一个输入框，输入 cmd 确定后，会打开命令行终端。

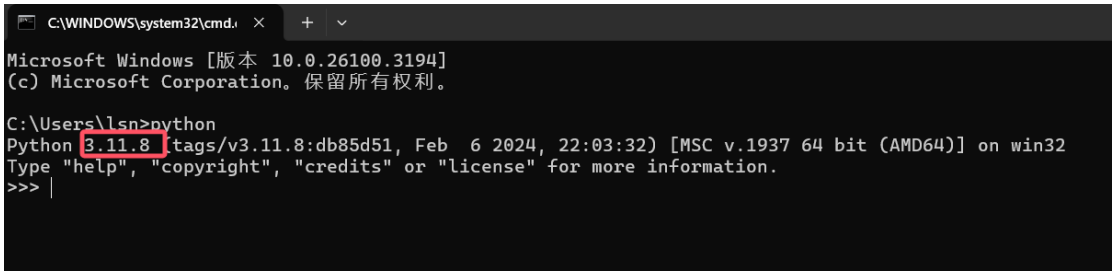




或者在搜索中输入 cmd 打开。


cmd
全部
应用
网页
设置
文件夹
最佳匹配
命令提示符
系统
应用
Git CMD
Developer Command Prompt for VS 2022
Anaconda Prompt
搜索网页
cmd - 查看更多搜索结果
cmd管理员身份运行 命令
文档
cmdoptions.cpython-311.pyc
_cmd.cpython-311.pyc
搜索

在终端输入 python，回车，终端会显示当前使用的 python 版本。



配置环境变量

若在终端输入 python 之后提示“不是内部或外部命令，也不是可运行的程序”，需要将 python.exe 配置到 PATH 环境变量中。

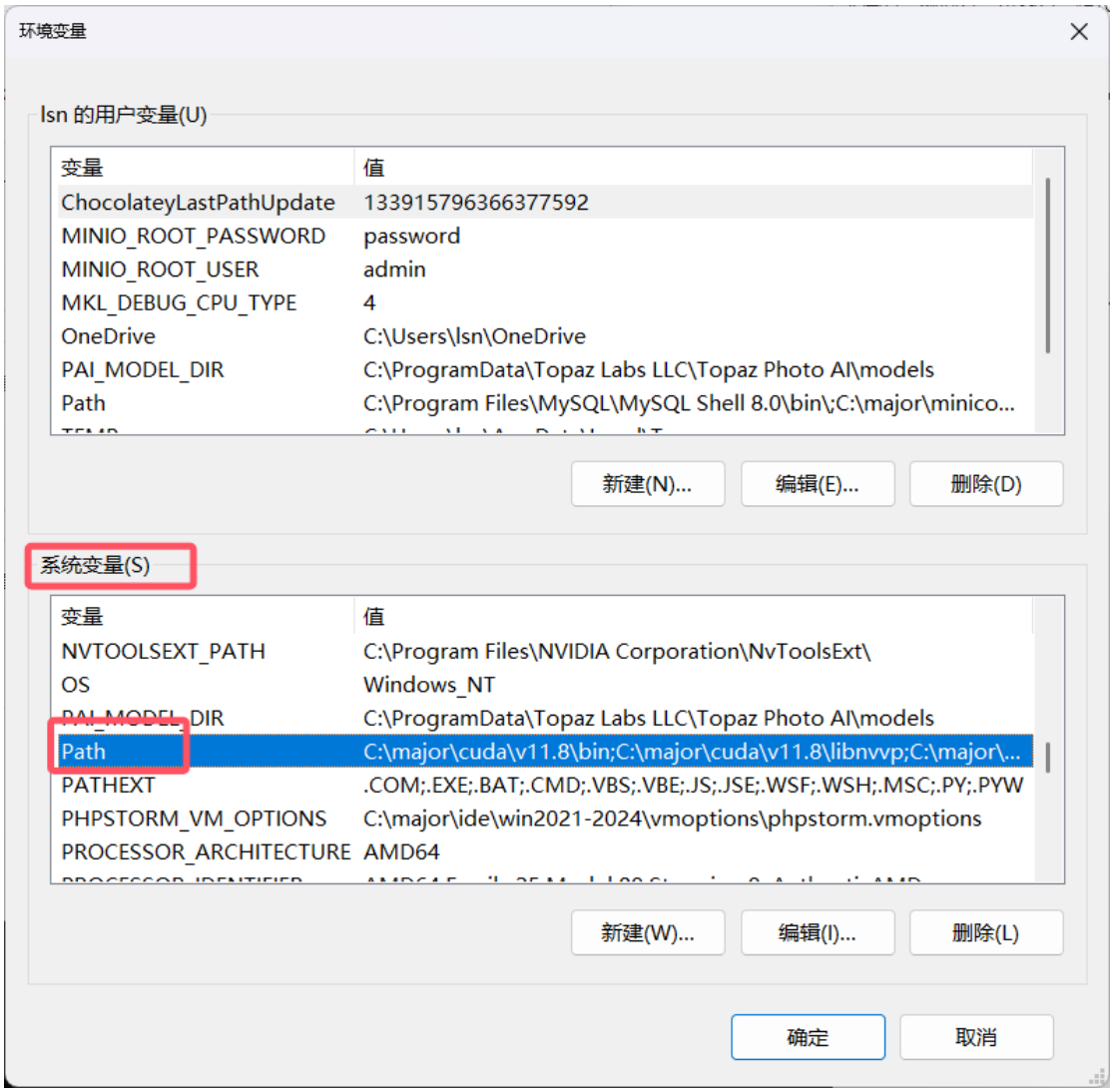
搜索“环境变量”，点击打开“编辑系统环境变量”



点击“环境变量”

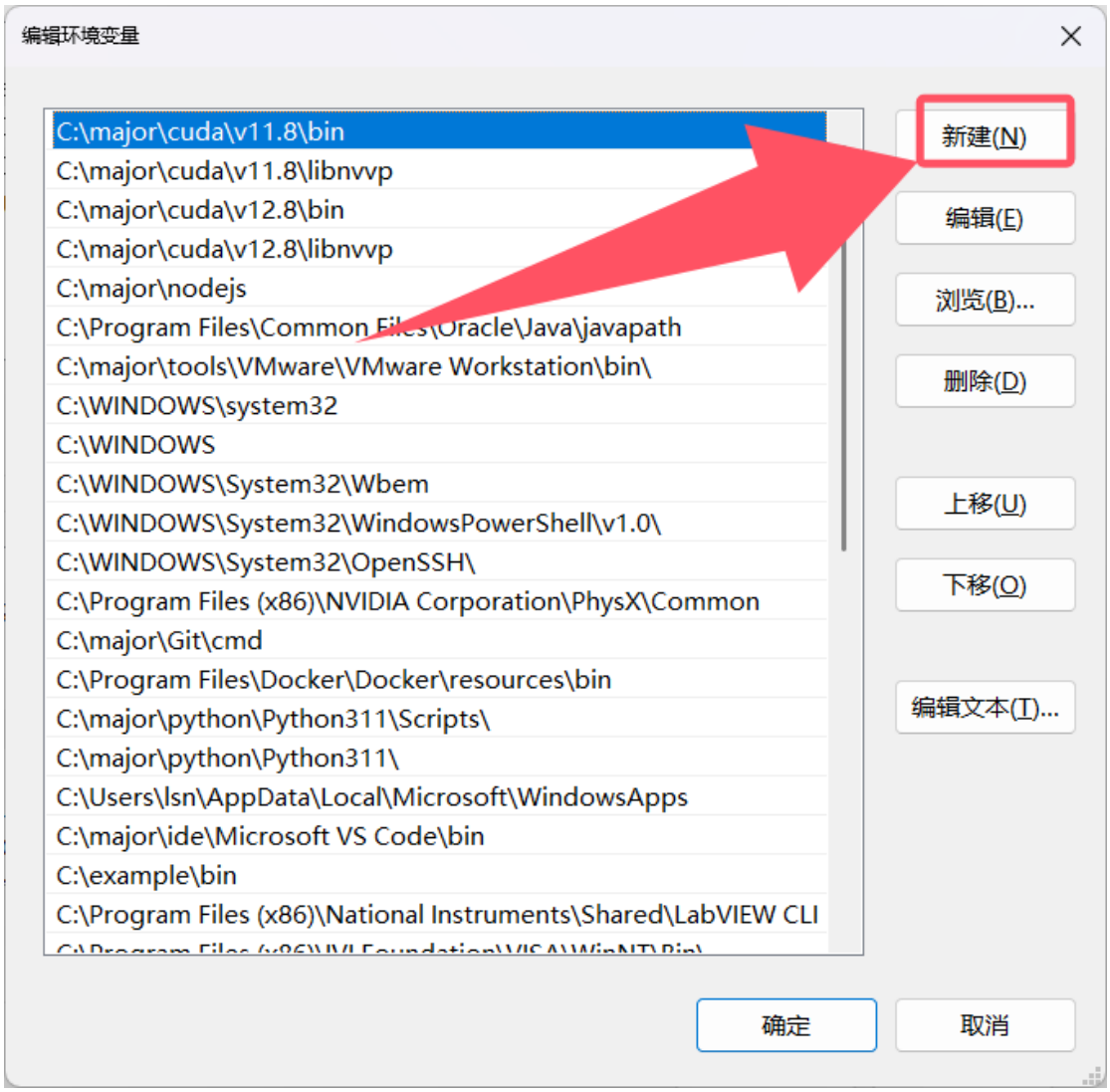


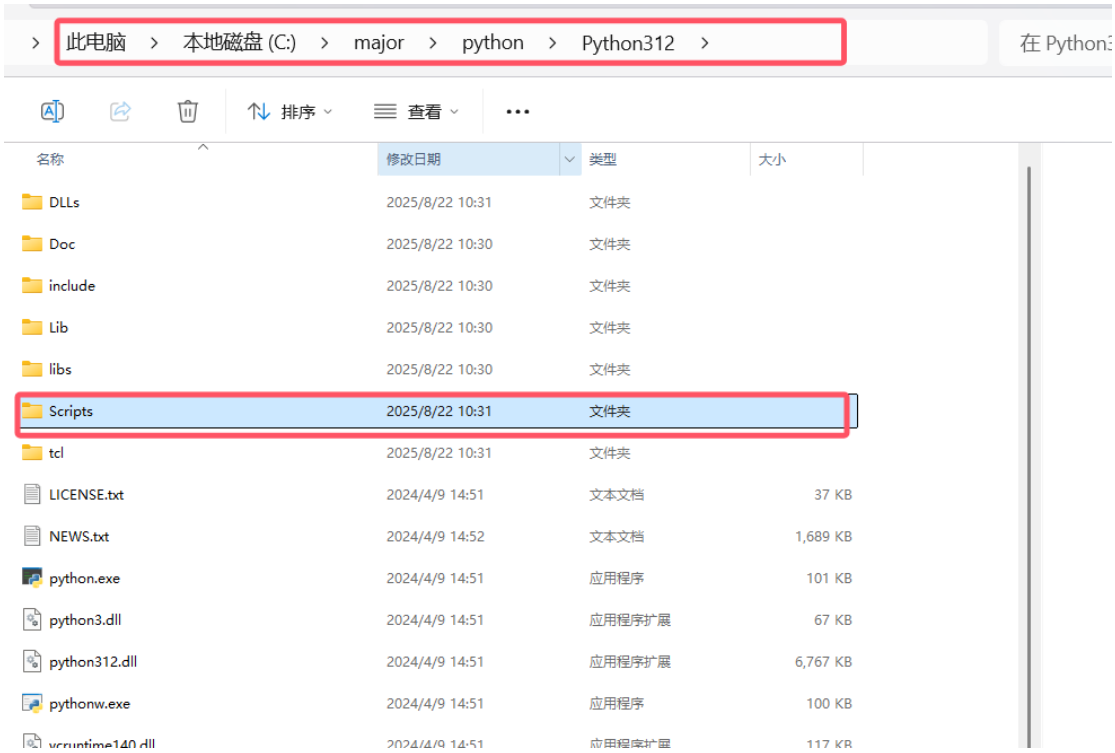
在“系统变量”中找到“Path”一行，点击编辑。

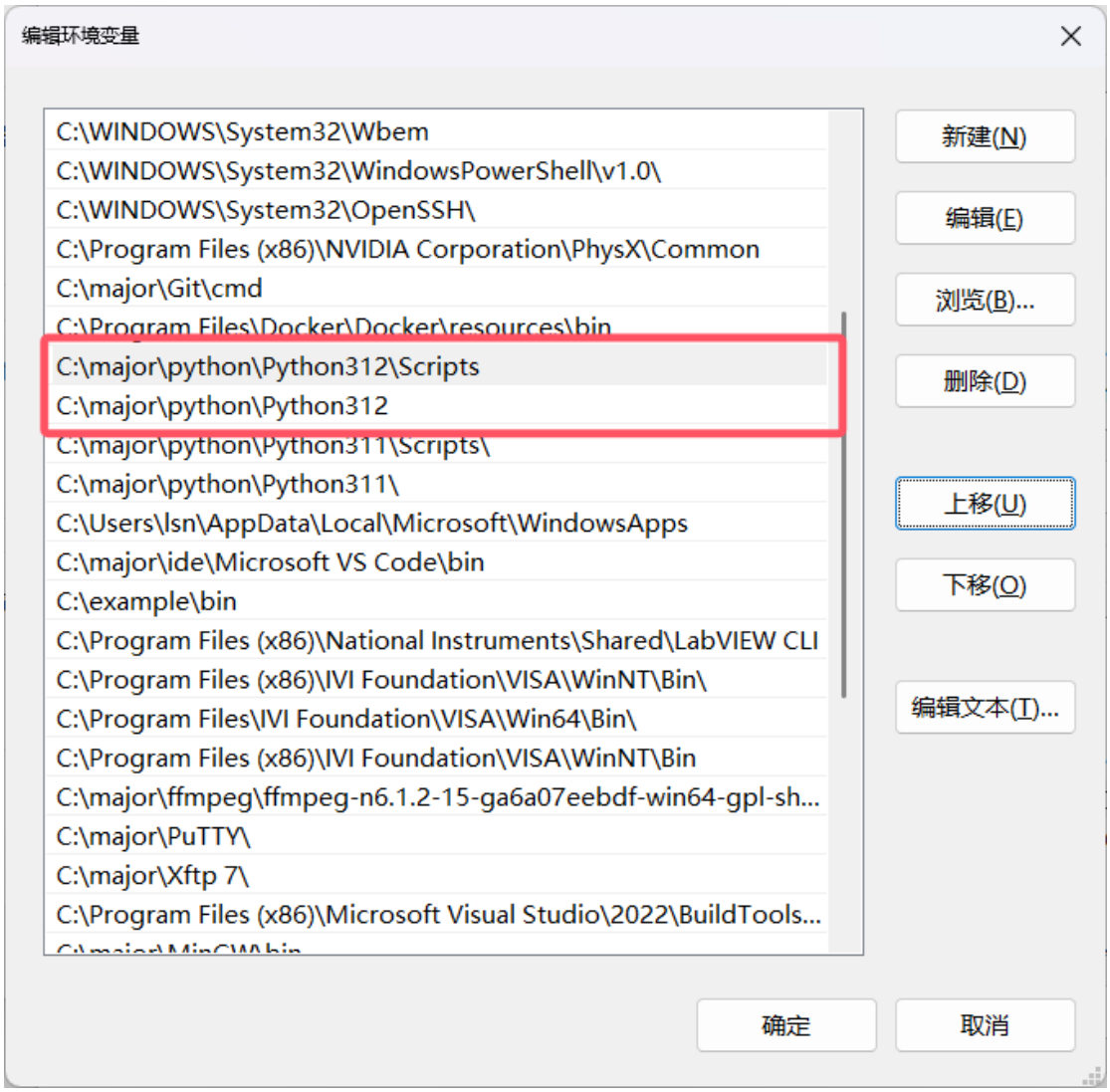


点击“新建”，找到 python 的安装路径，以及该路径下的 Scripts 文件夹，把这两个文件夹的完整路径分别填到新建的变量中。

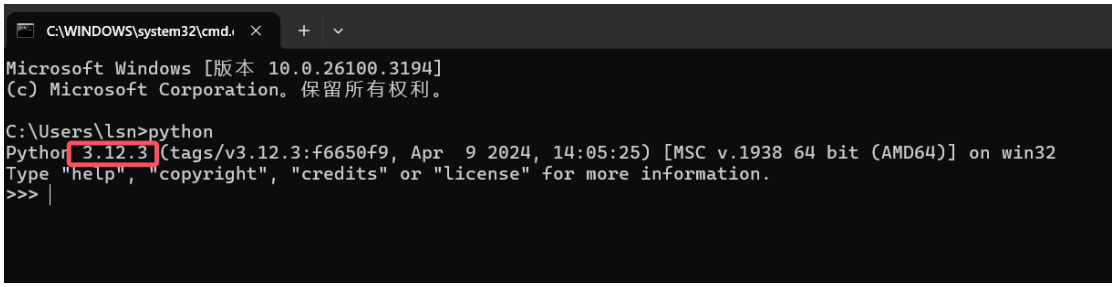
可添加多个 python，点击右侧的“上移”“下移”移动变量的位置，优先使用在上面的变量。


编辑环境变量
C:\major\cuda\v11.8\bin
C:\major\cuda\v11.8\libnvvp
C:\major\cuda\v12.8\bin
C:\major\cuda\v12.8\libnvvp
C:\major\nodejs
C:\Program Files\Common Files\Oracle\Java\javapath
C:\major\tools\VMware\VMware Workstation\bin\
C:\WINDOWS\system32
C:\WINDOWS
C:\WINDOWS\System32\Wbem
C:\WINDOWS\System32\WindowsPowerShell\v1.0\
C:\WINDOWS\System32\OpenSSH\
C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
C:\major\Git\cmd
C:\Program Files\Docker\Docker\resources\bin
C:\major\python\Python311\Scripts\
C:\major\python\Python311\
C:\Users\lsn\AppData\Local\Microsoft\WindowsApps
C:\major\ide\Microsoft VS Code\bin
C:\example\bin
C:\Program Files (x86)\National Instruments\Shared\LabVIEW CLI
新建(N)
编辑(E)
浏览(B)...
删除(D)
上移(U)
下移(O)
编辑文本(T)...
确定
取消


此电脑 › 本地磁盘 (C:) › major › python › Python312 ›
在 Python3
排序
查看
名称
修改日期
类型
大小
DLLs 2025/8/22 10:31 文件夹
Doc 2025/8/22 10:30 文件夹
include 2025/8/22 10:30 文件夹
Lib 2025/8/22 10:30 文件夹
libs 2025/8/22 10:30 文件夹
Scripts 2025/8/22 10:31 文件夹
tcl 2025/8/22 10:31 文件夹
LICENSE.txt 2024/4/9 14:51 文本文档 37 KB
NEWS.txt 2024/4/9 14:52 文本文档 1,689 KB
python.exe 2024/4/9 14:51 应用程序 101 KB
python3.dll 2024/4/9 14:51 应用程序扩展 67 KB
python312.dll 2024/4/9 14:51 应用程序扩展 6,767 KB
pythonw.exe 2024/4/9 14:51 应用程序 100 KB



点击确定。打开终端再次输入 python，即可显示版本信息。



2. 创建虚拟环境

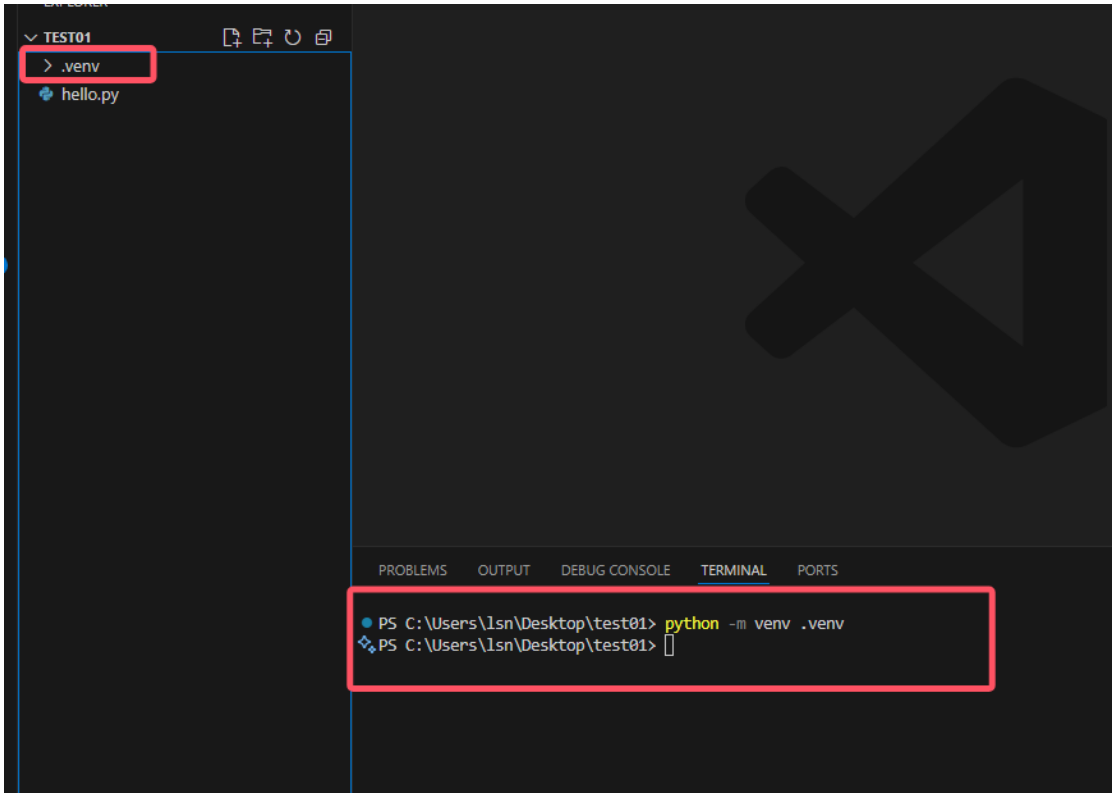
后续使用 conda 进行管理虚拟环境。

不同版本的 python、第三方库会依赖不同的库，直接使用 pip 会在全局 python 环境中安装库，这会导致不同版本库之间的兼容问题，所以建议为每一个项目单独创建一个虚拟环境，在虚拟环境中安装所需的库。

进入项目目录，在终端输入：

python -m venv .venv

创建虚拟环境



激活虚拟环境

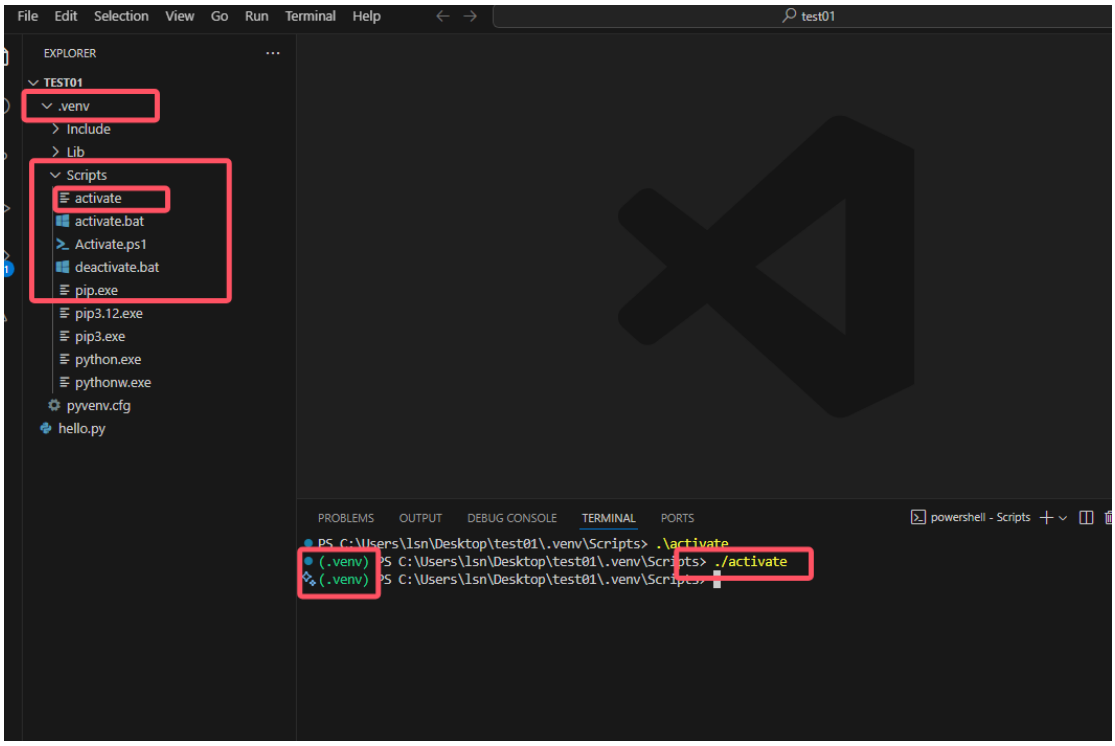
终端进入到.venv/Scripts 目录，输入

./activate

Windows + PowerShell 下

.\.venv\Scripts\Activate.ps1

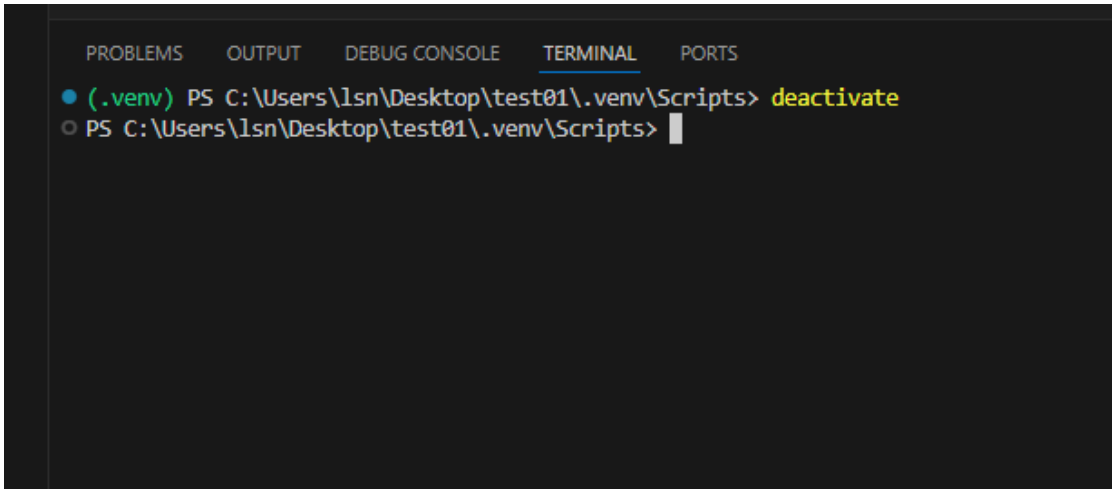
激活环境，激活后，路径前面会显示虚拟环境名称。



输入

deactivate

取消激活环境。



Anaconda

我们主要使用其中的 conda 来进行虚拟环境管理，以避免与机器人现有环境冲突。

下载

官网，进入官网仓库 下载 Windows 和 Linux 两个版本，演示电脑为 windows 系统，树莓派 5 为 arm 架构。

下载的两个版本如下图所示：

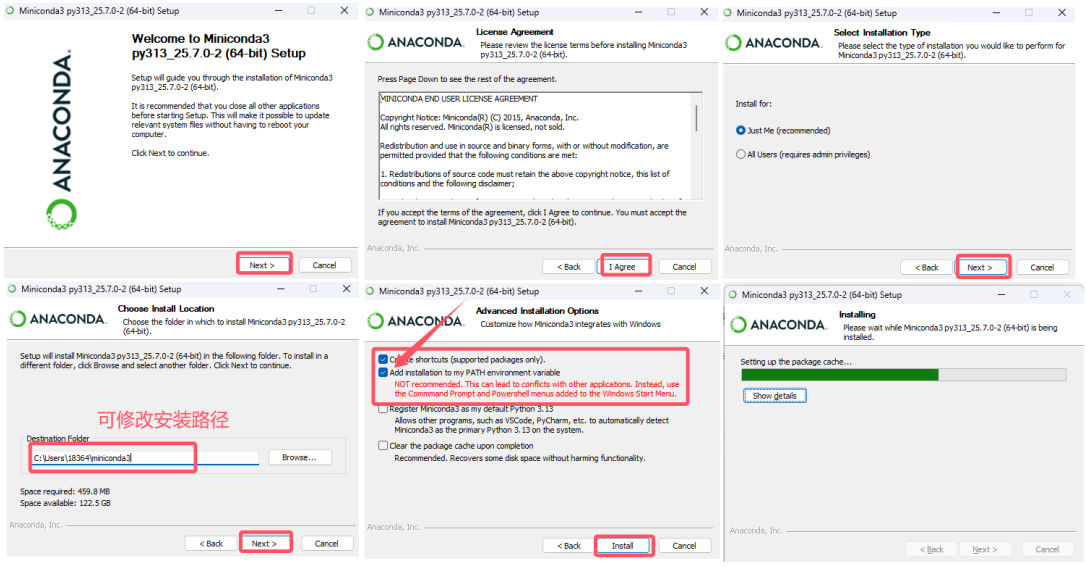
Miniconda3-py311_25.7.0-2-Windows-x86_64.exe



安装

Windows

点击打开下载完成的安装包，一步步安装即可。



验证安装

终端输入：

conda --version

或者

conda -V

输出以下内容说明安装成功：

C:\Users\lsn>conda -V

conda 25.5.1

C:\Users\lsn>

创建虚拟环境

conda create -n myvenv01 python=3.11

列出已有虚拟环境

conda env list

激活环境

conda activate myvenv01

命令行输入

python Hello.py

取消激活

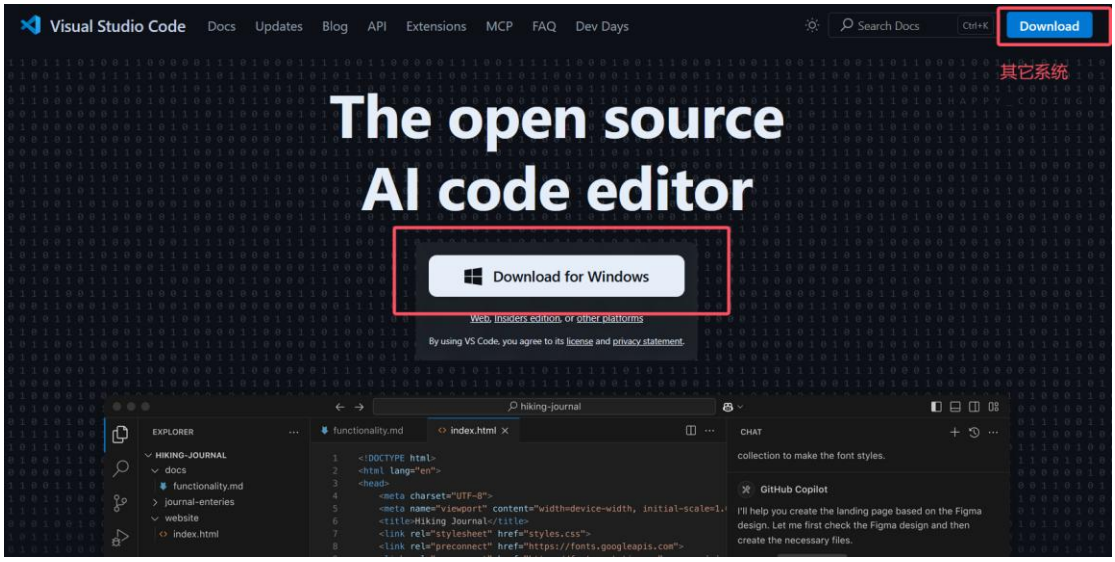
conda deactivate

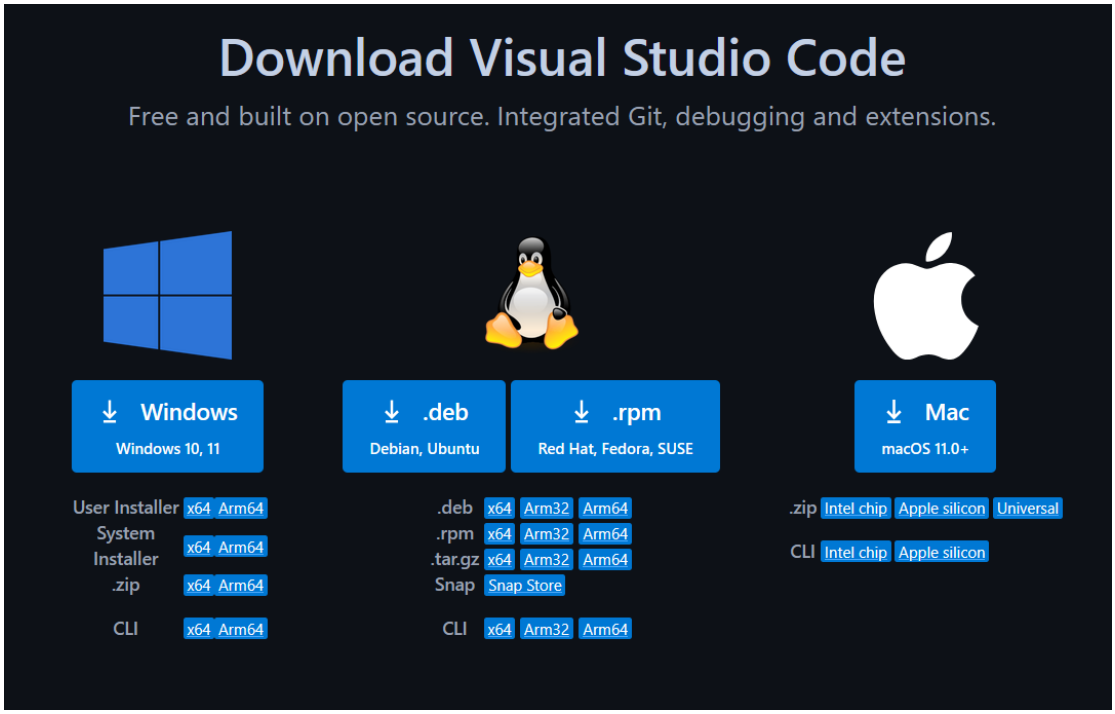
VSCode 安装

Visual Studio code 是微软开发的一款 IDE 集成开发环境软件，后续课程的代码演示使用该软件。也可以使用其它软件如 pycharm、cursor、trae 等。

下载

进入 vscode 官网，直接点击下载即可，其它系统点击右上方的下载，选择合适的版本。



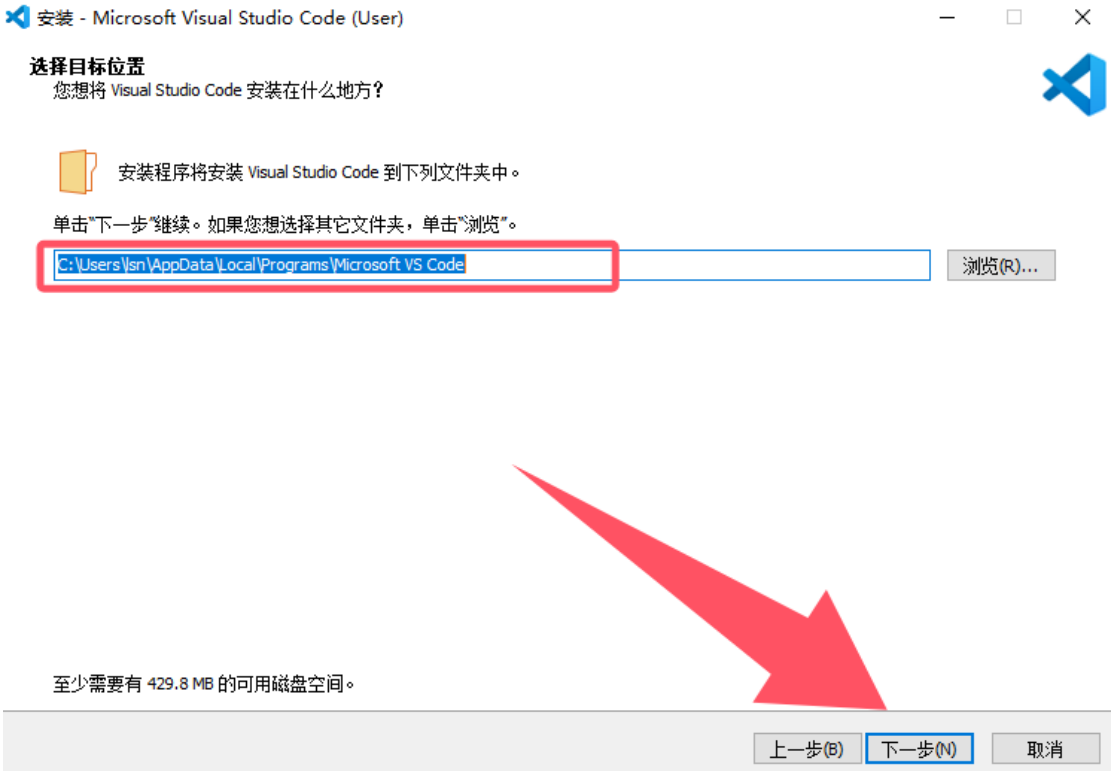


安装

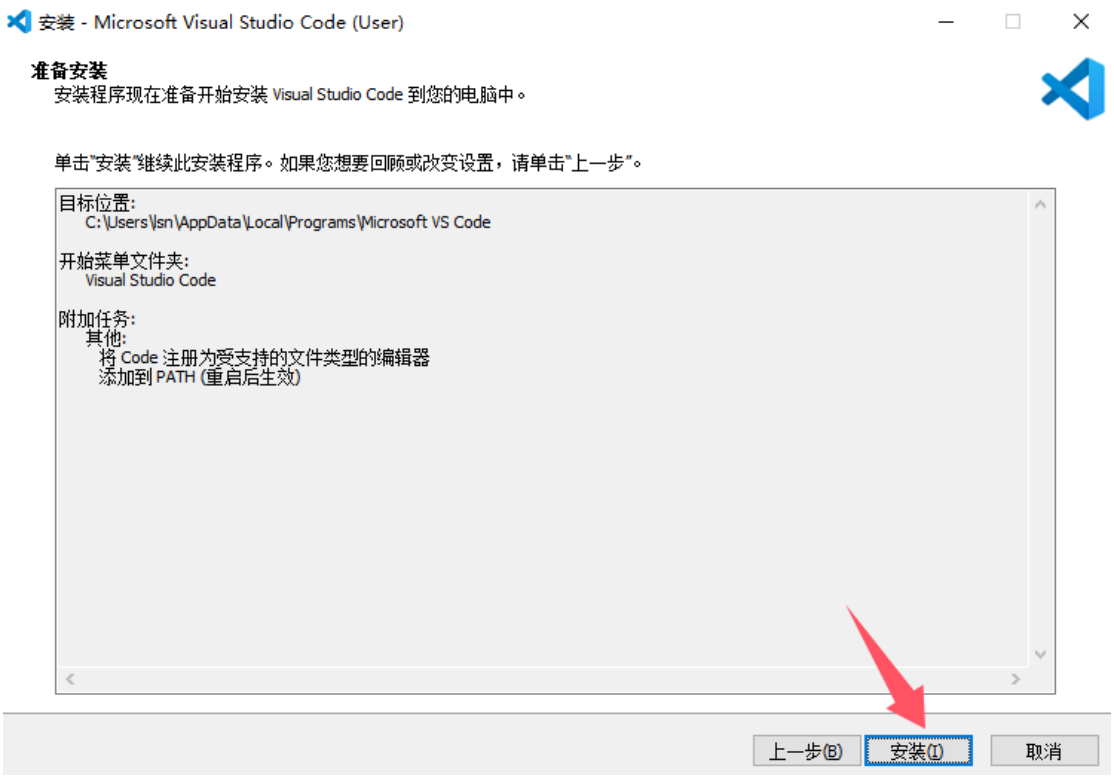
下载完成之后，点击安装。点击同意，下一步。



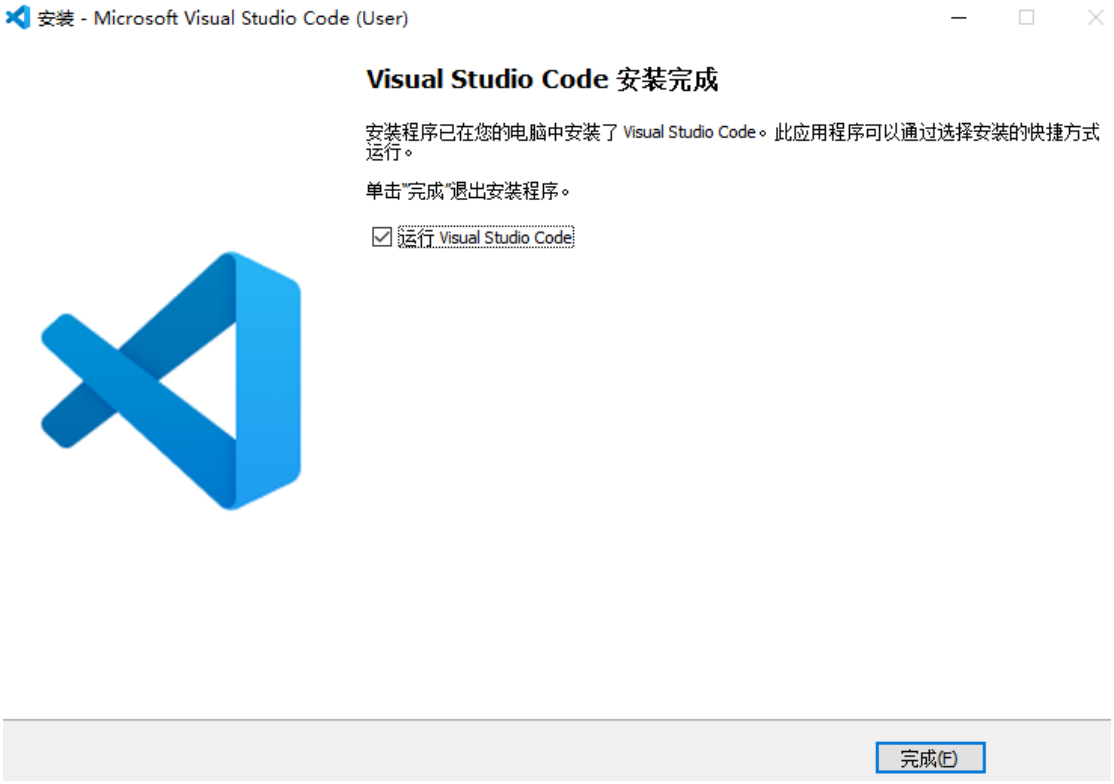
选择安装位置，点击下一步。后续选项保持默认即可，点击下一步。

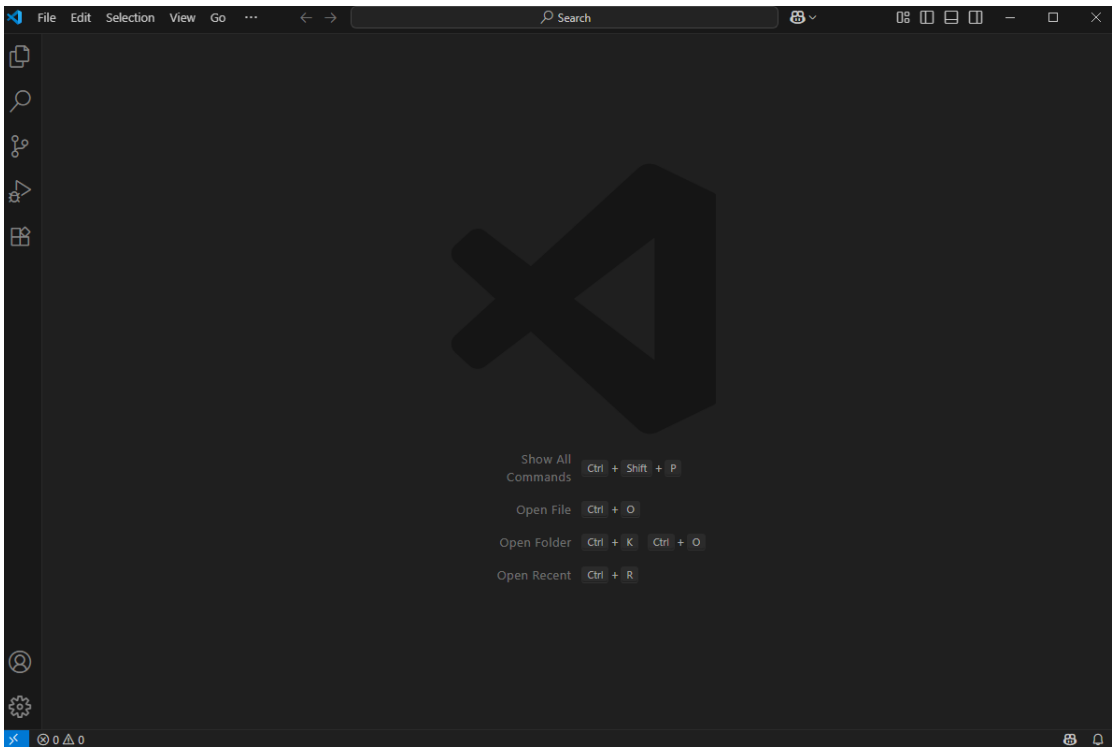


点击安装。


安装 - Microsoft Visual Studio Code (User)
准备安装
安装程序现在准备开始安装 Visual Studio Code 到您的电脑中。
单击"安装"继续此安装程序。如果您想要回顾或改变设置，请单击"上一步"。
目标位置:
C:\Users\lsn\AppData\Local\Programs\Microsoft VS Code
开始菜单文件夹:
Visual Studio Code
附加任务:
其他:
将 Code 注册为受支持的文件类型的编辑器
添加到 PATH (重启后生效)
上一步(B)
安装(I)
取消

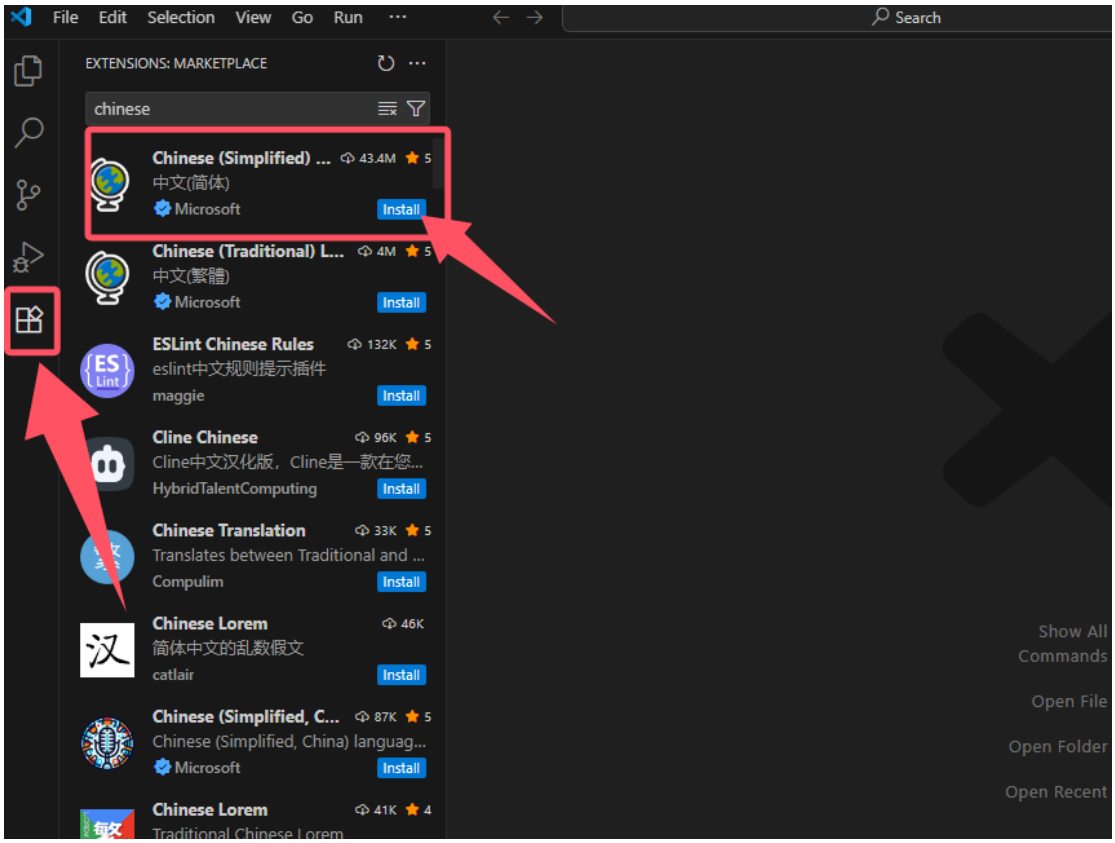
点击完成。


安装 - Microsoft Visual Studio Code (User)
Visual Studio Code 安装完成
安装程序已在您的电脑中安装了 Visual Studio Code。此应用程序可以通过选择安装的快捷方式运行。
单击"完成"退出安装程序。
运行 Visual Studio Code
完成(F)



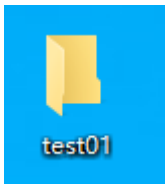
语言设置

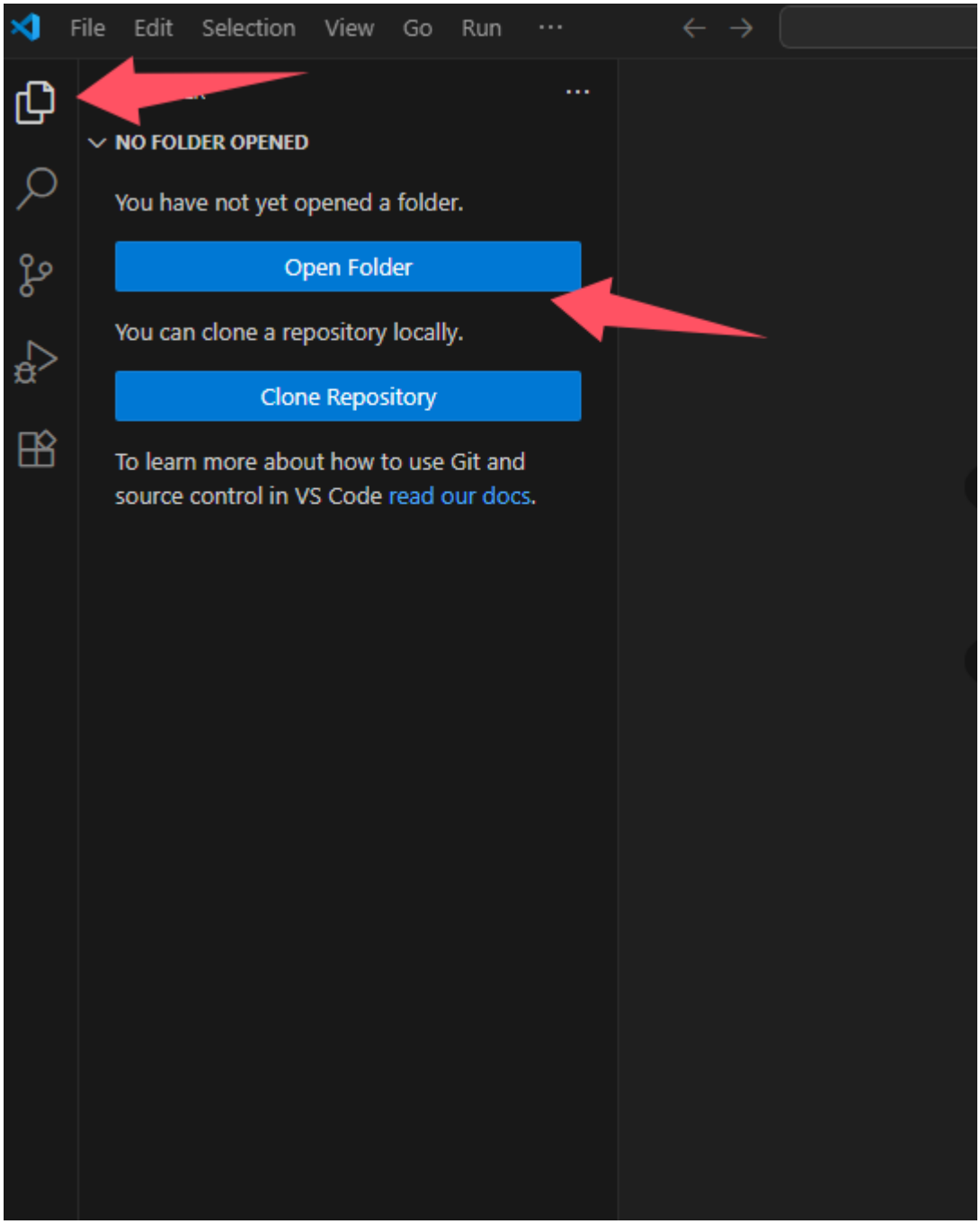
打开 vscode，点击左侧的插件按钮，输入 chinese，可安装汉化插件，安装完成后重启即可。

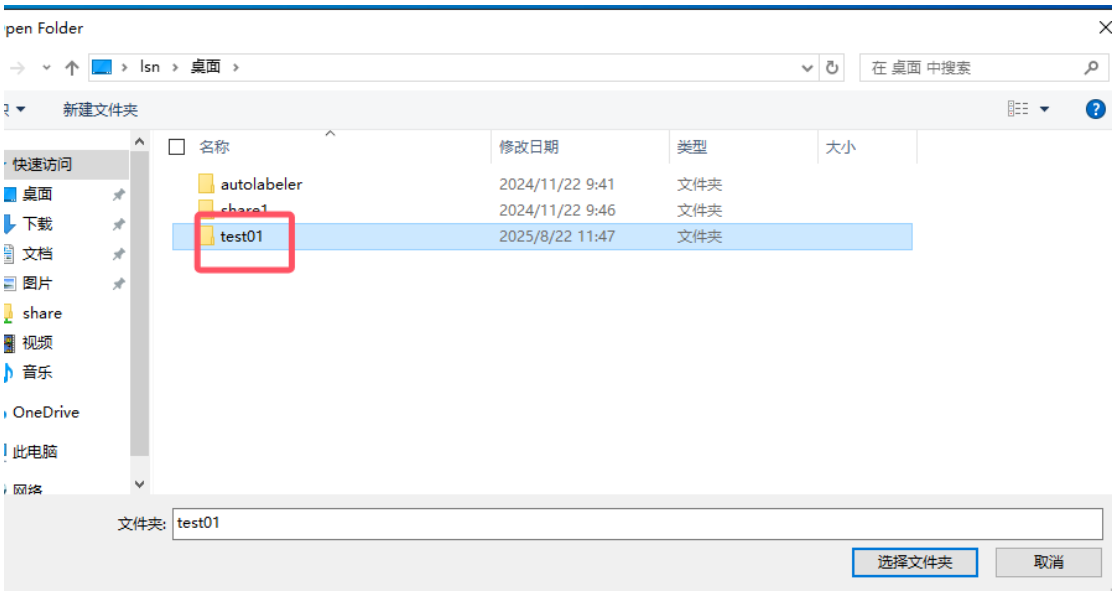


创建文件夹

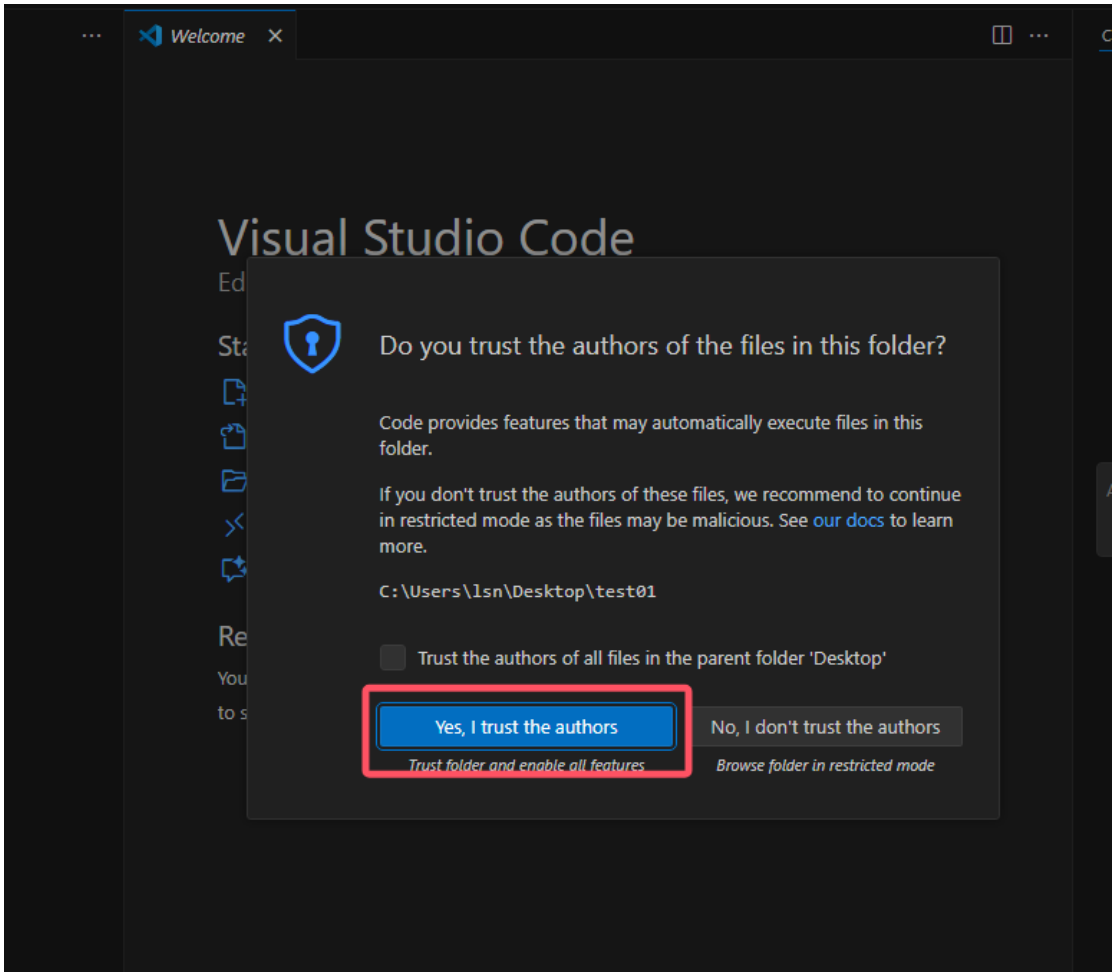
在电脑上创建一个文件夹，打开 vscode，点击左上方的文件按钮，点击打开文件夹，选择刚才创建的文件夹打开。



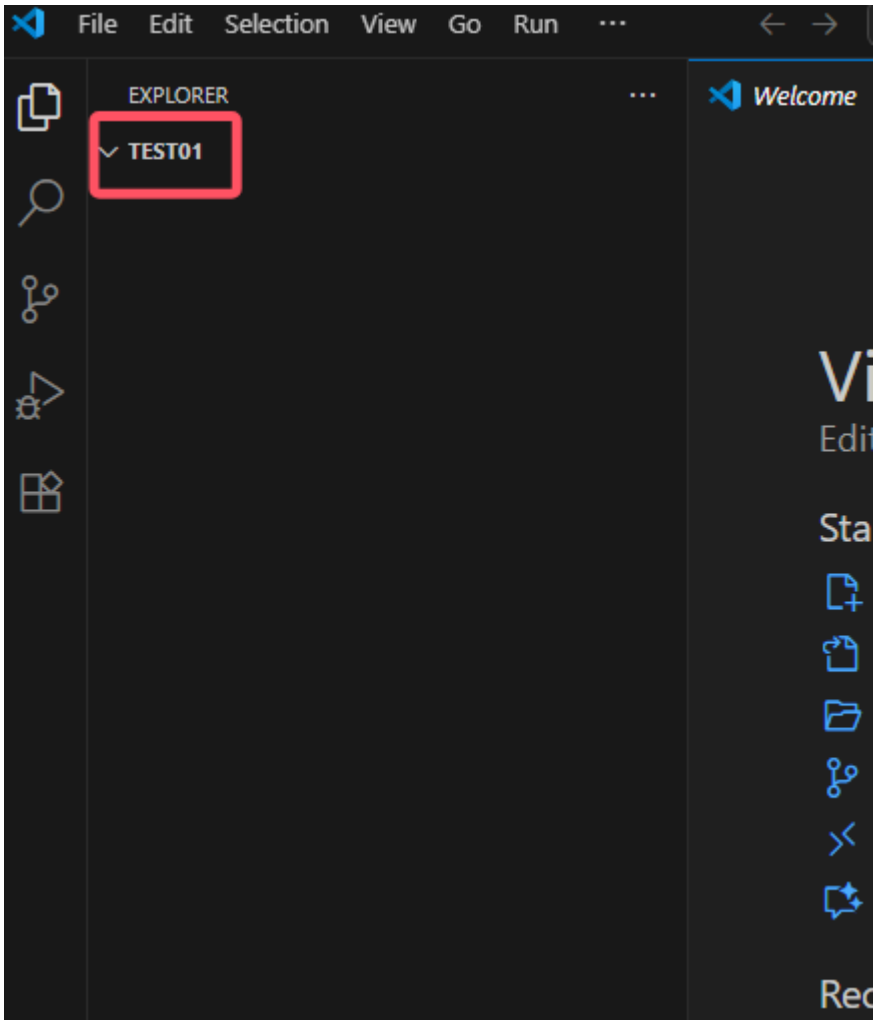




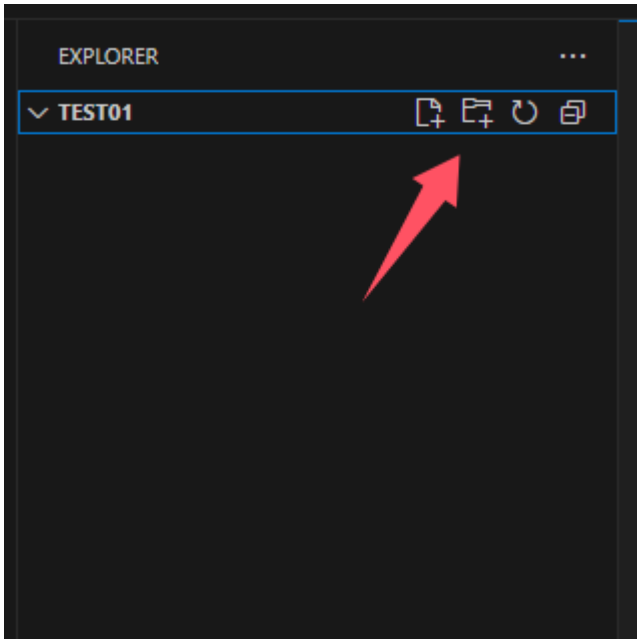
点击信任作者。



打开文件夹后，左上方会显示文件夹名称。

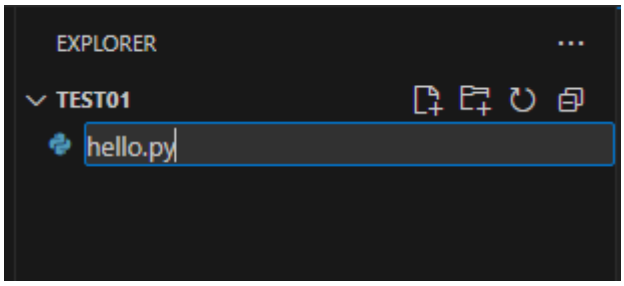


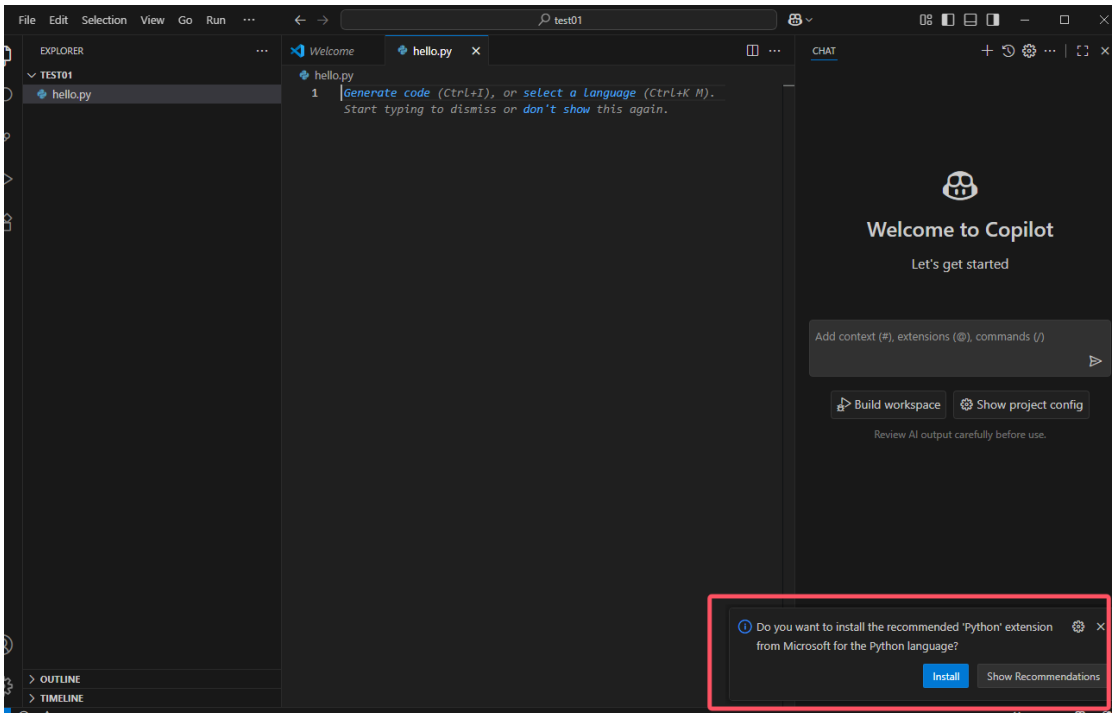
点击文件夹名称，右侧会显示操作图标，可以创建文件、文件夹等。

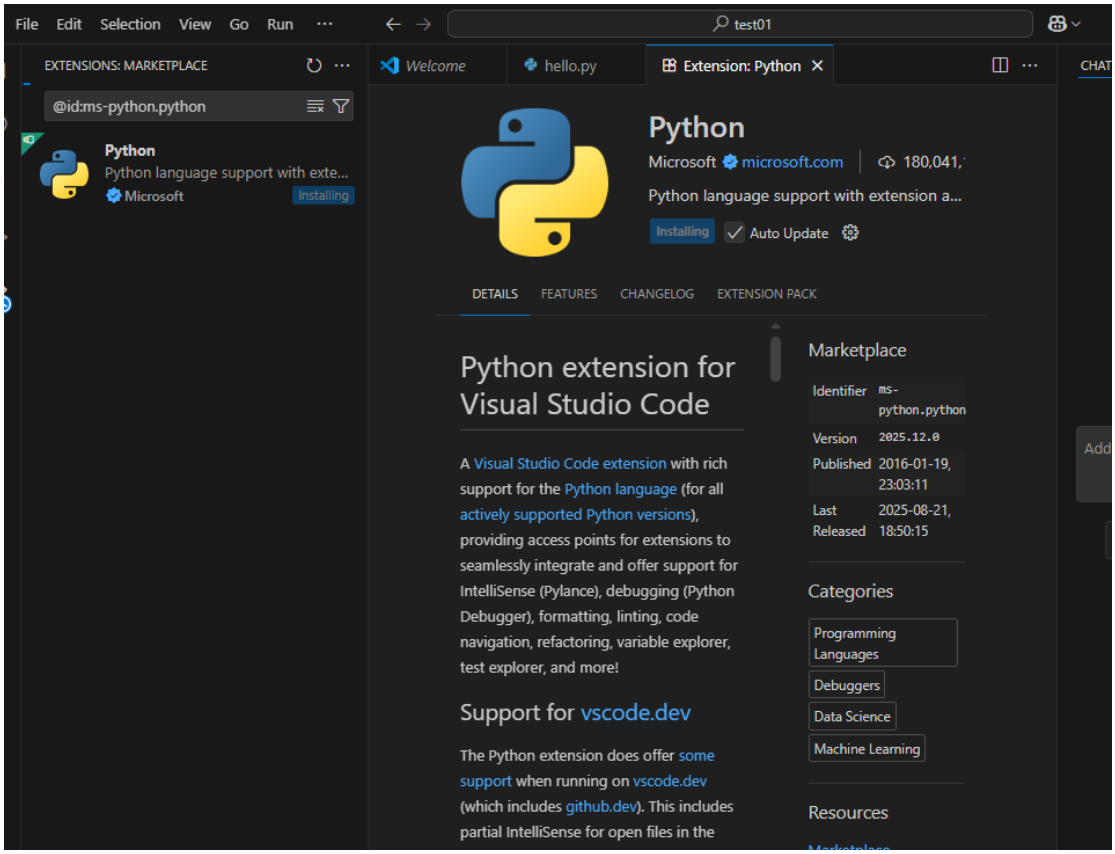


安装 python 插件

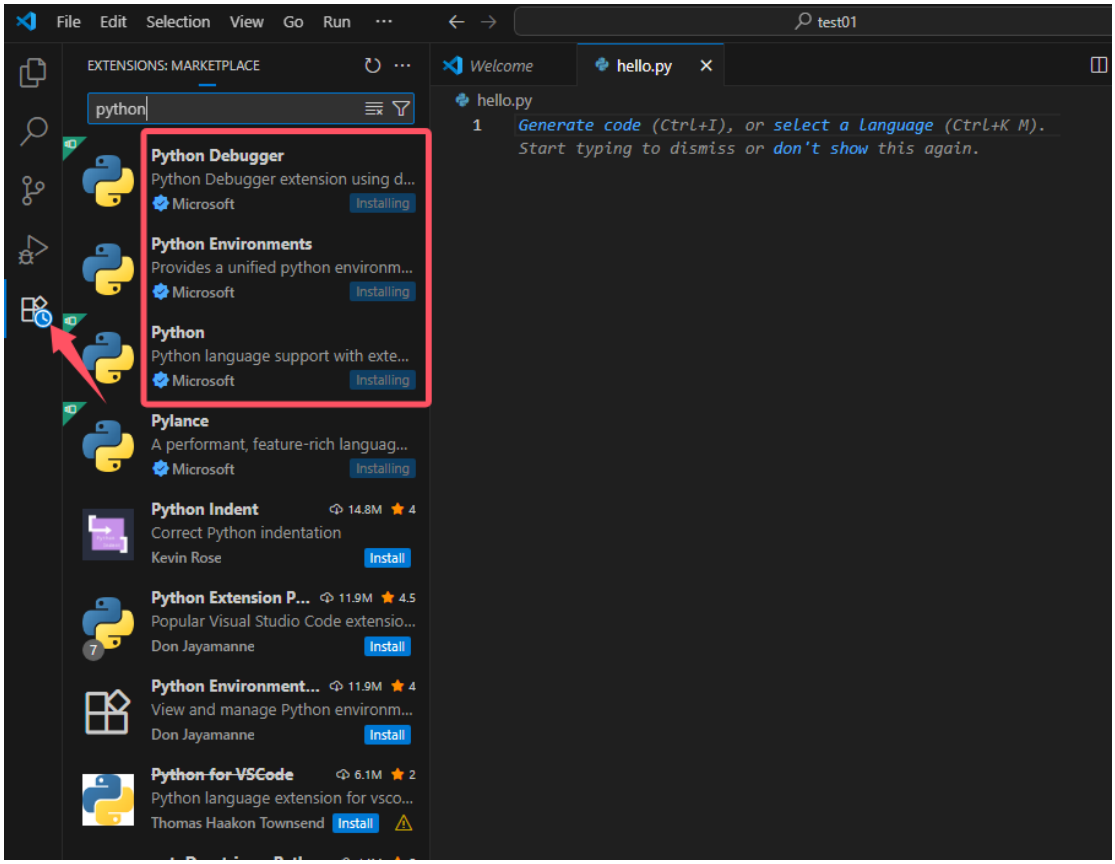
创建一个 hello.py 文件，打开，右下角会出现下载 python 插件的提示，直接安装即可。



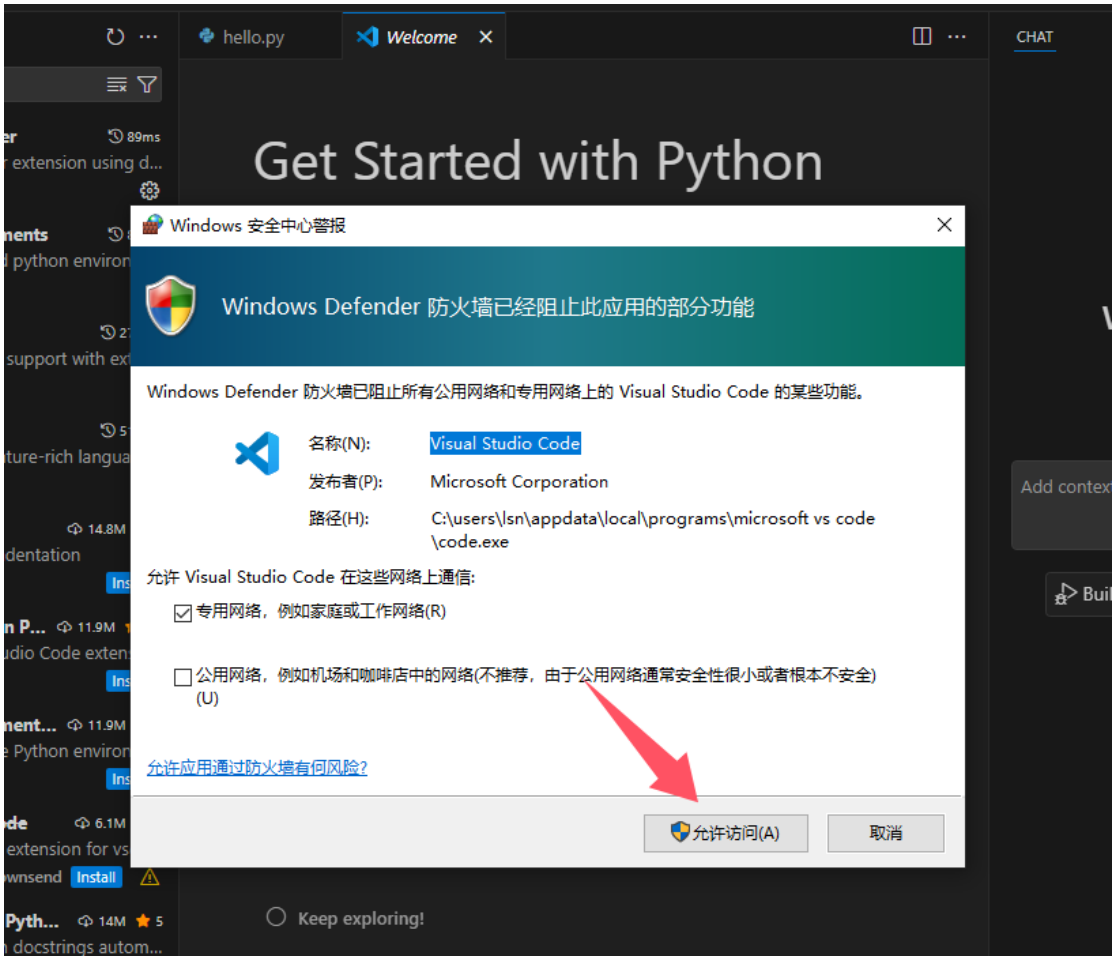




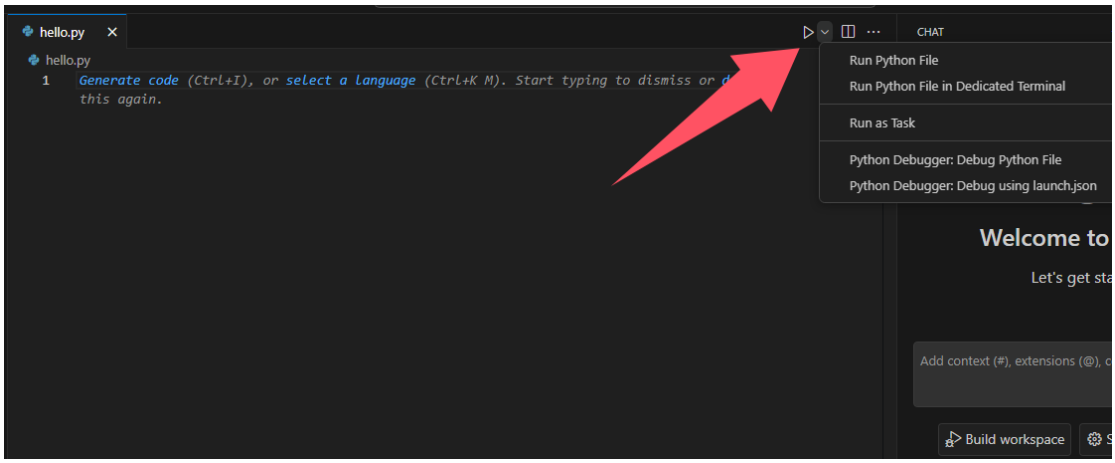
如果没有出现提示，直接在插件中搜索 python，手动安装 Python Debugger、Python Environments、Python 即可。


File Edit Selection View Go Run ···
test01
EXTENSIONS: MARKETPLACE
python
Python Debugger
Python Debugger extension using d...
Microsoft
Installing
Python Environments
Provides a unified python environm...
Microsoft
Installing
Python
Python language support with exte...
Microsoft
Installing
Pylance
A performant, feature-rich languag...
Microsoft
Installing
Python Indent
14.8M
4
Correct Python indentation
Kevin Rose
Install
Python Extension P...
11.9M
4.5
Popular Visual Studio Code extensio...
Don Jayamanne
Install
Python Environment...
11.9M
4
View and manage Python environm...
Don Jayamanne
Install
Python for VSCode
6.1M
2
Python language extension for vsco...
Thomas Haakon Townsend
Install
Welcome
hello.py
hello.py
1
Generate code (Ctrl+I), or select a language (Ctrl+K M).
Start typing to dismiss or don't show this again.

点击允许访问。



安装完成后，打开 hello.py 文件，右上角会显示一个三角形。

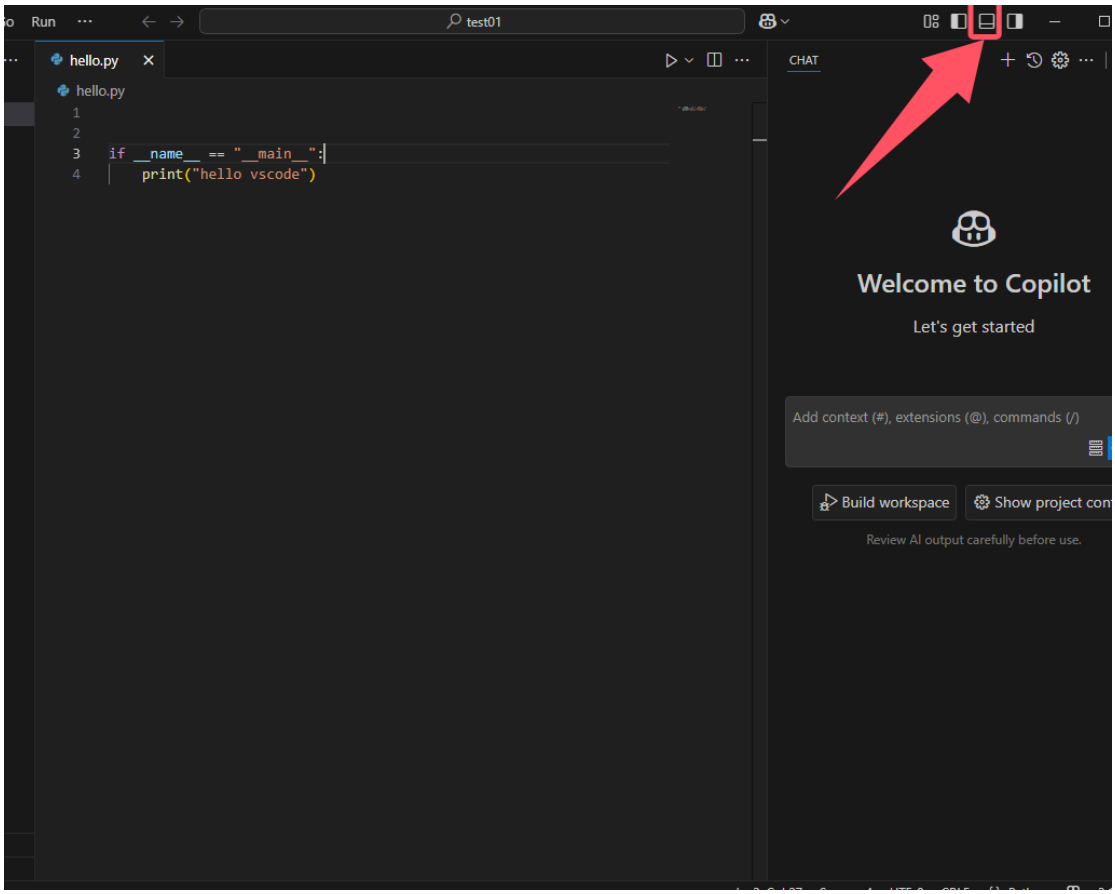


运行 python 程序

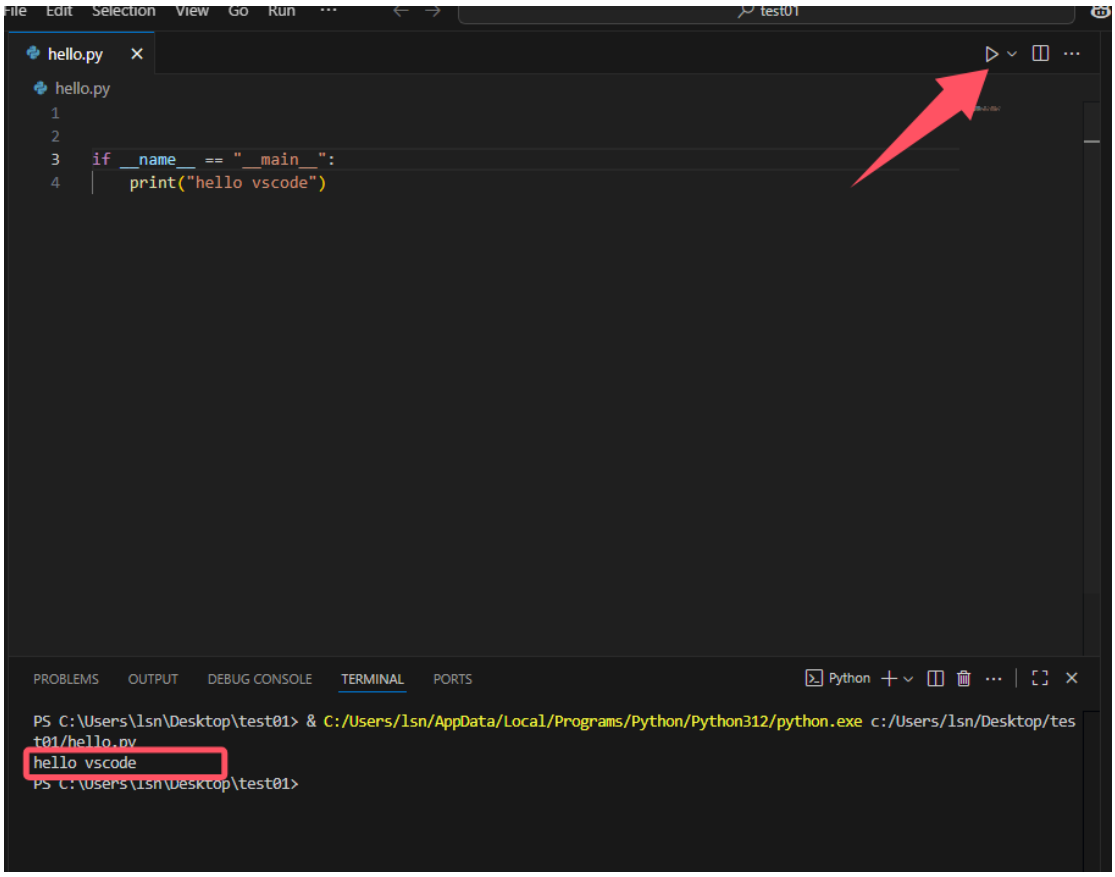
在 hello.py 文件中编写示例代码。

```
if __name__ == "__main__":
    print("hello vscode")
```

点击右上角面板显示按钮，可以显示/隐藏终端。



点击右上角允许该程序，下方的终端会输出"hello vscode"。



观察终端输出的上一行:

C:/Users/lsn/AppData/Programs/Python/Python312/python.exe c:/Users/lsn/Desktop/teste1/hello.py

- ...python.exe ： 这个就是之前安装的 python
- ...hello.py ： 这个是我们编写的程序

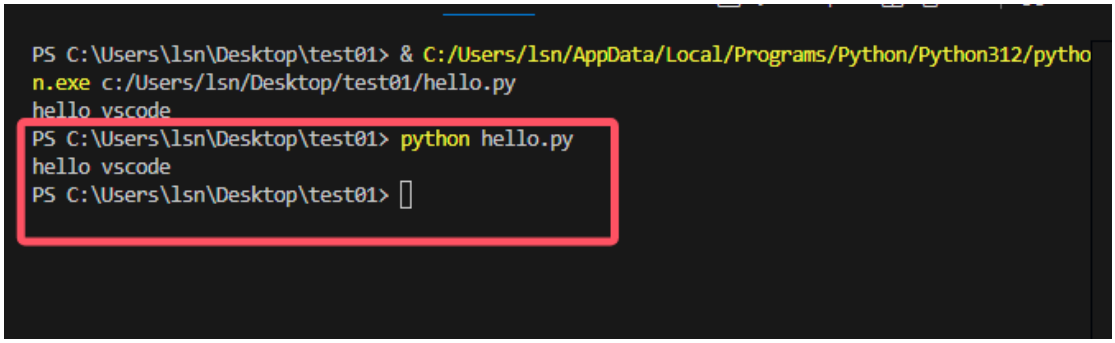
这个命令就是终端执行 python 程序的命令

python xxx.py

因为我们添加了 python 的环境变量，所以可以不输入完整的 python 路径。

在终端输入:

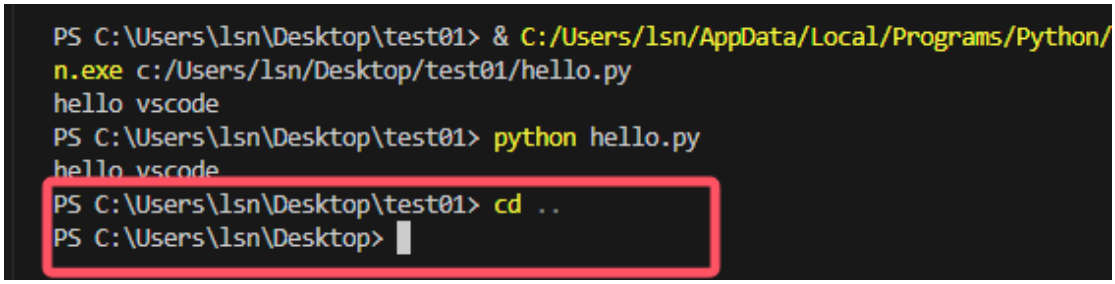
python hello.py



可以看到程序正常运行了。不过 python 后面的 hello.py 文件也不是完整路径，因为当前终端所在的路径是...\test01，也就是 hello.py 文件所在的路径，那么执行命令的时候，python 会在当前文件寻找 hello.py 文件进行执行，所以不需要完整路径。

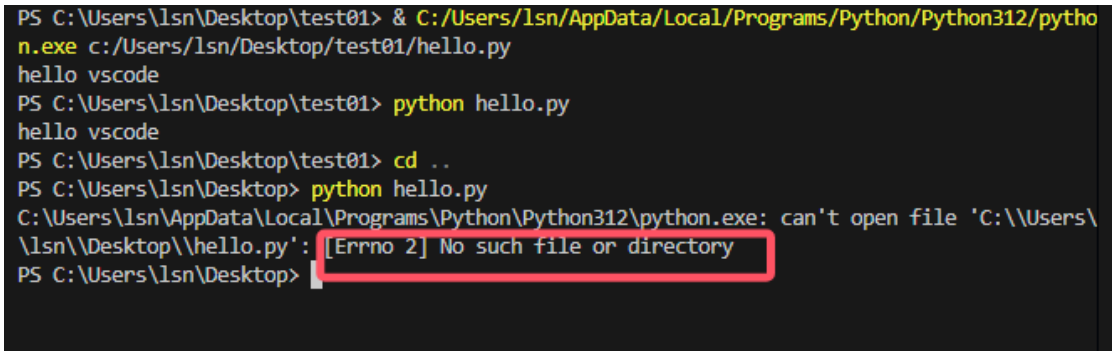
我们在终端输入：

cd ..



可以看到当前路径变成了 test01 的上一级目录。

我们再次执行 hello.py 文件。



可以看到执行出现错误：没有这个文件或者目录。

我们在终端输入 ls 来查看当前目录下的文件和文件夹：

ls

```
PS C:\Users\lsn\Desktop> ls


    目录: C:\Users\lsn\Desktop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        2024/11/22      9:41                autolabeler
d-----        2024/11/22      9:46                share1
d-----         2025/8/22     11:52                test01
-a----         2022/2/17     10:37         982328 FastCopy.exe
-a----         2025/8/22     11:11       64701312 Git-2.51.0-64-bit.exe
-a----        2024/11/21     22:54           2336 Microsoft Edge.lnk
-a----         2025/8/22     10:24       26712328 python-3.12.3-amd64.exe
-a----         2025/8/22     11:12      114897568 VSCodeUserSetup-x64-1.103.2.exe


PS C:\Users\lsn\Desktop>
```

可以看到，当前目录下确实没有 hello.py 这个文件，所以执行错误。

那么，我们在终端输入：

python ./test01/hello.py

```
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        2024/11/22      9:41                autolabeler
d-----        2024/11/22      9:46                share1
d-----         2025/8/22     11:52                test01
-a----         2022/2/17     10:37         982328 FastCopy.exe
-a----         2025/8/22     11:11       64701312 Git-2.51.0-64-bit.exe
-a----        2024/11/21     22:54           2336 Microsoft Edge.lnk
-a----         2025/8/22     10:24       26712328 python-3.12.3-amd64.exe
-a----         2025/8/22     11:12      114897568 VSCodeUserSetup-x64-1.103.2.exe


PS C:\Users\lsn\Desktop> python .\test01\hello.py
hello vscode
PS C:\Users\lsn\Desktop>
```

可以看到程序再次成功运行。

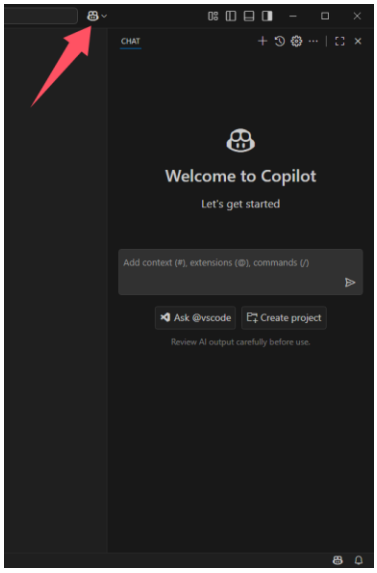
综上，在终端运行 python 程序时，你可以 cd 到程序所在的目录下后，

cd C:\Users\bruce\Desktop\Test01

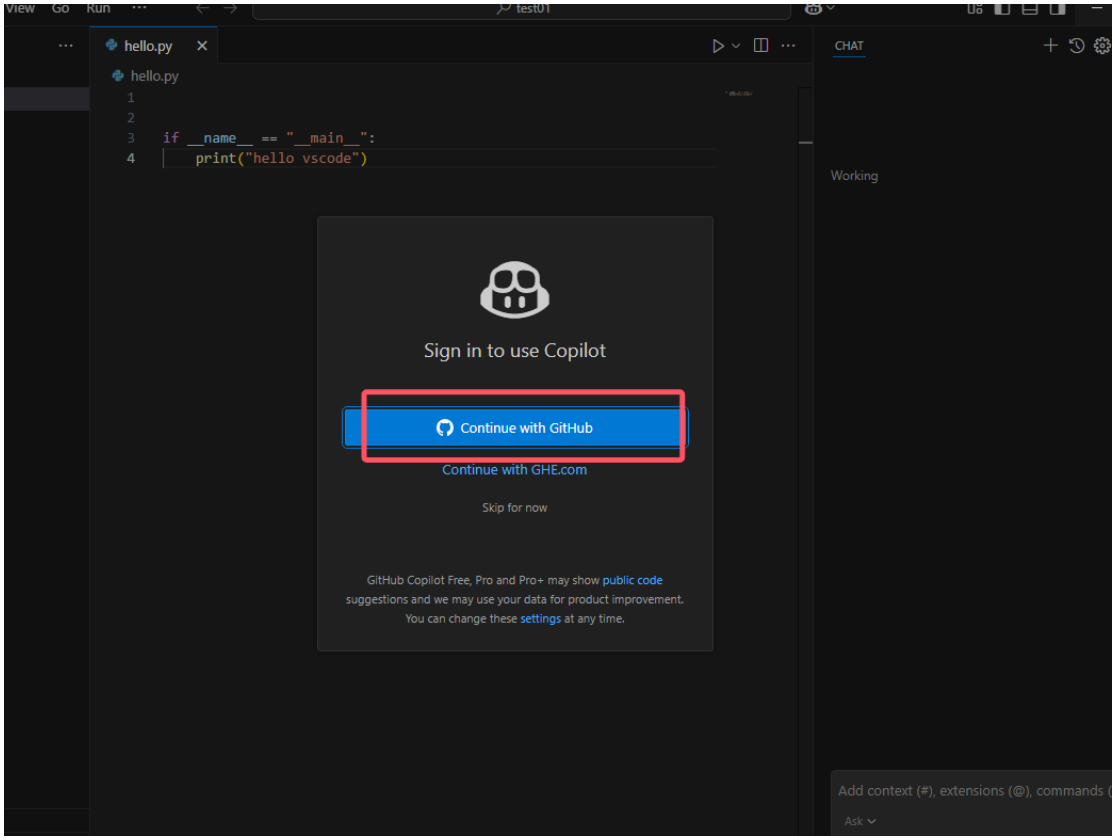
执行程序；或者输入完整/相对路径来执行。

Copilot

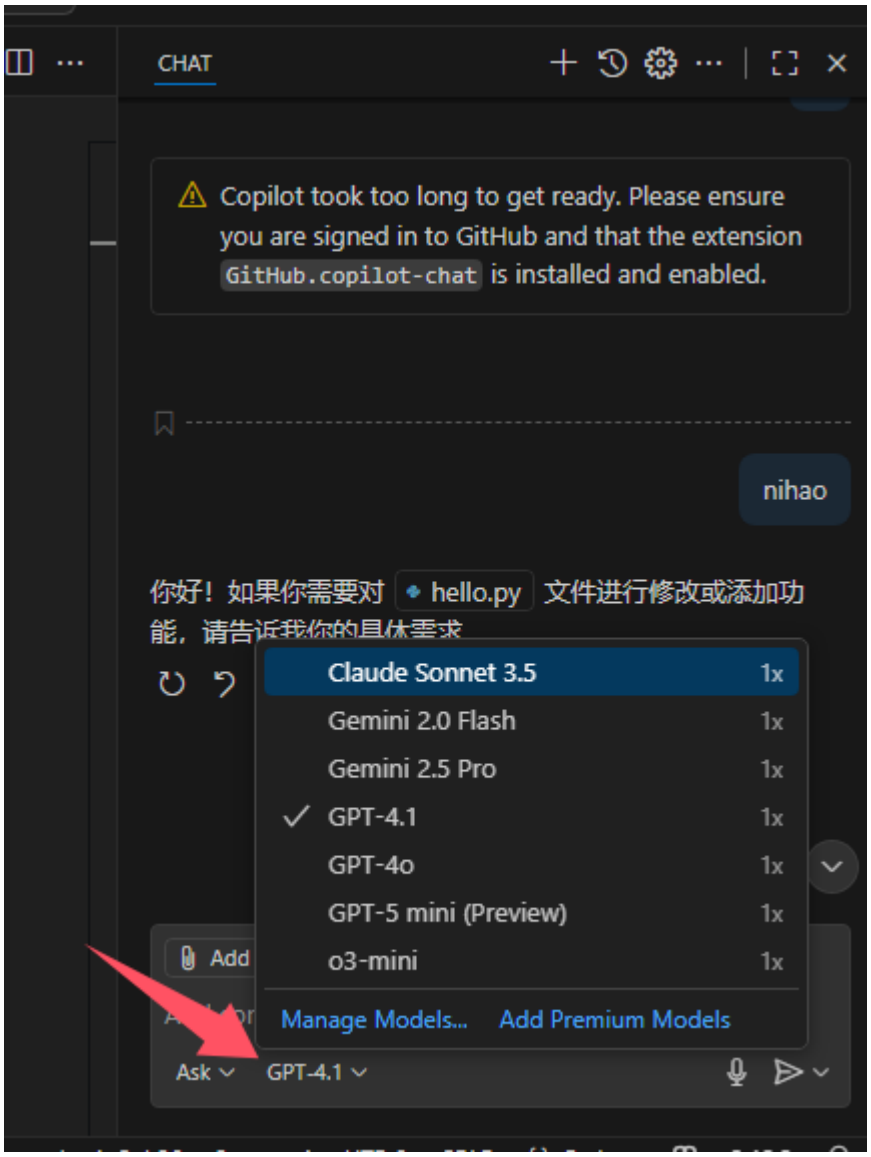
copilot 是 vscode 自带的一个 AI 插件，打开 vscode 点击右上方的机器人图标，即可打开对话框。



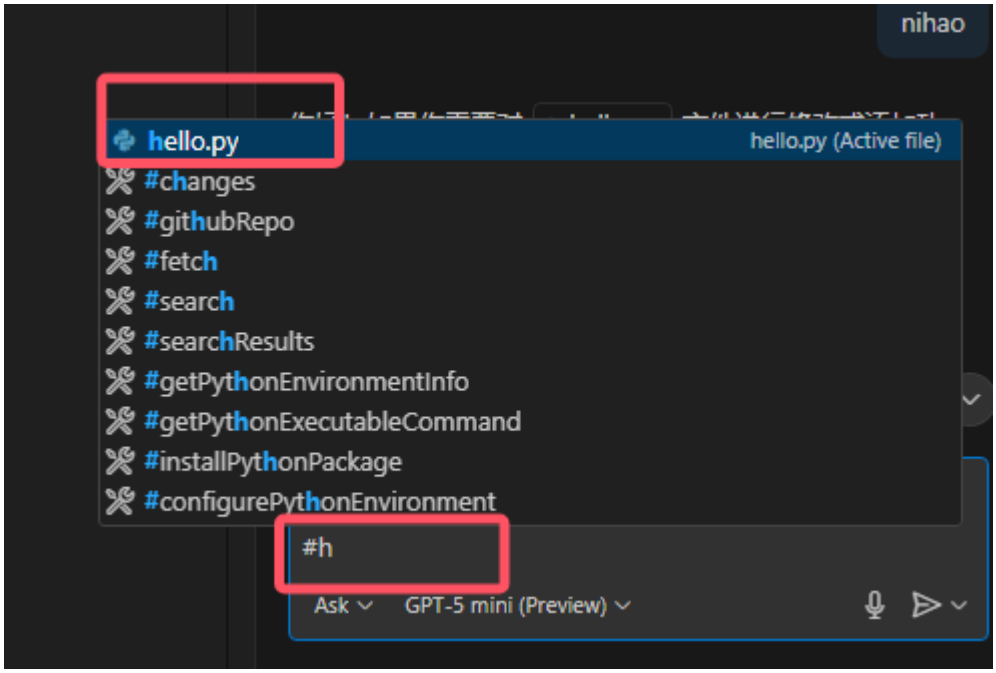
使用 github 登录 copilot。

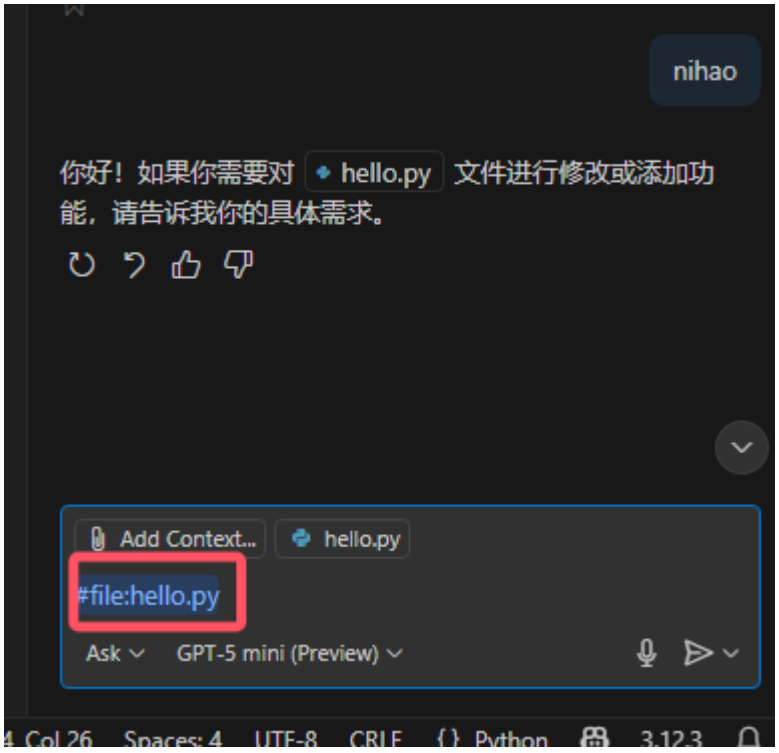


登录后可在输入框输入文本进行对话，对话框下方可以选择模型。



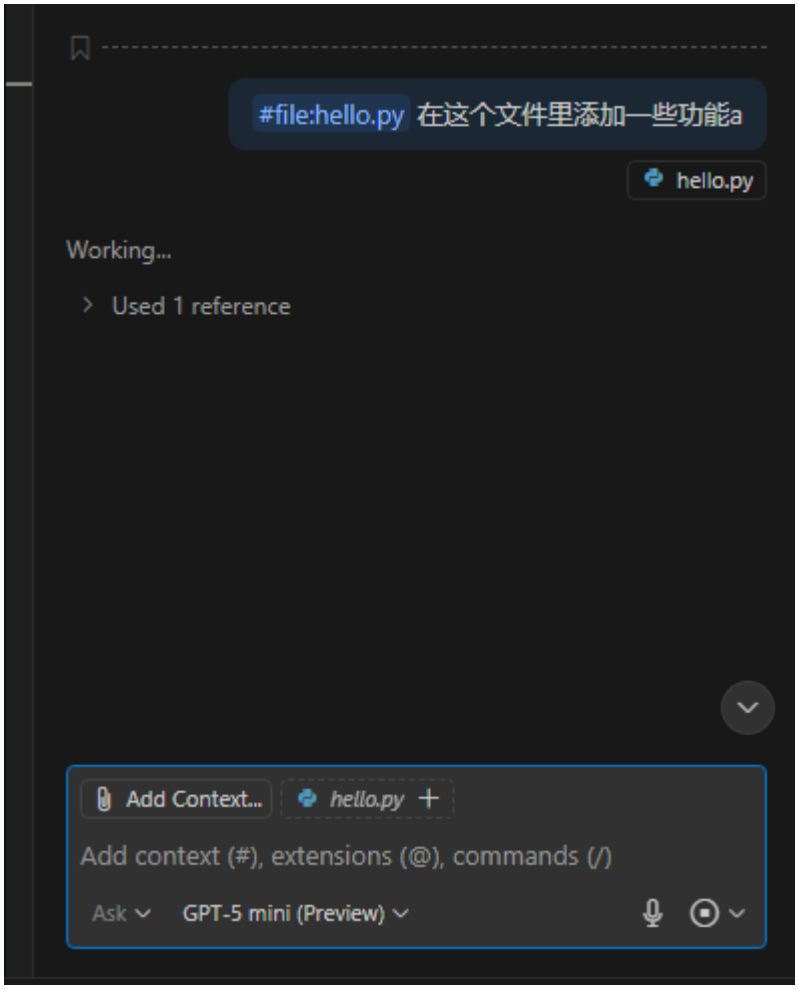
输入#hello，copilot 会匹配对应的文件，选择需要的文件可以将其添加为上下文。



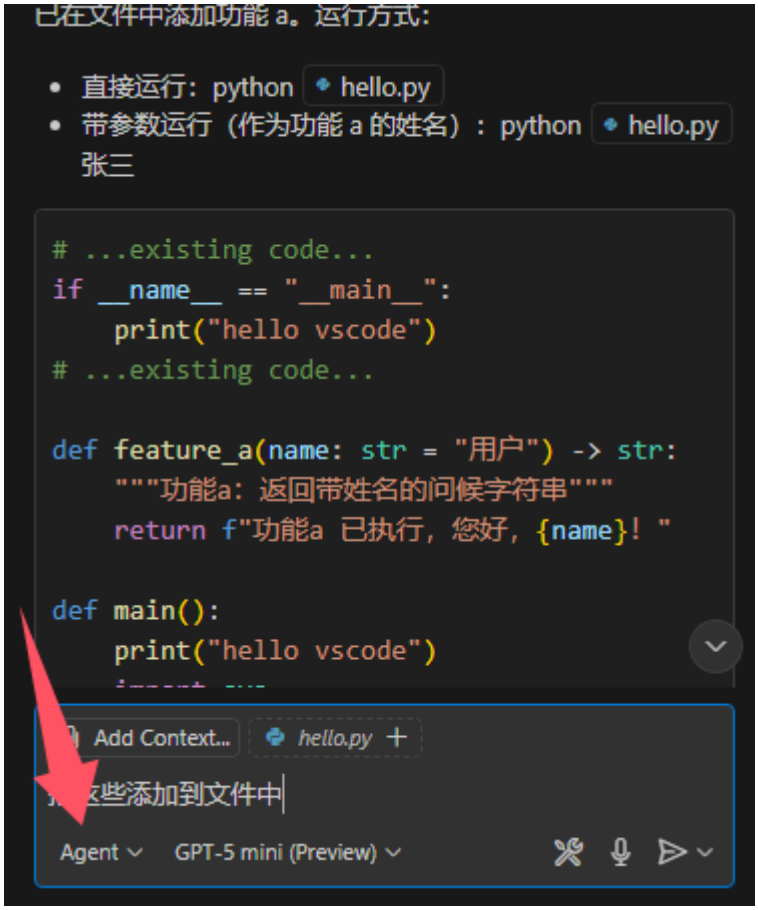


输入文本让它添加功能。

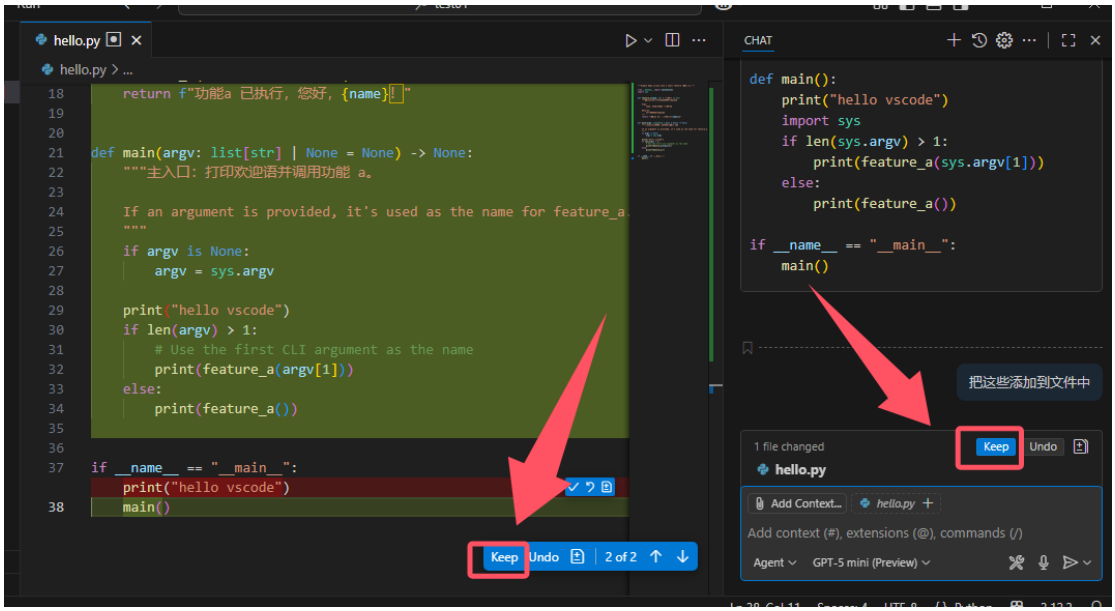




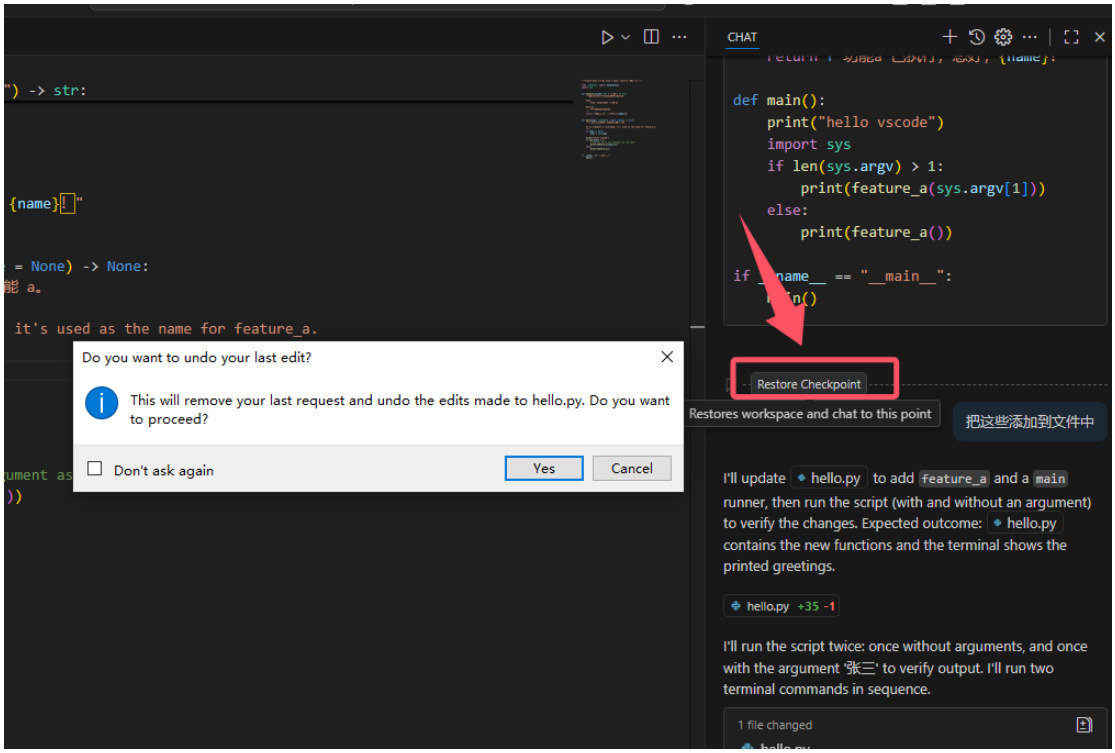
模型输出了结果，但是没有添加到文件中，因为当前的模式是“Ask”（对话框左下角），将模式改为 Edit/Agent 后，让它把新内容添加到文件中。

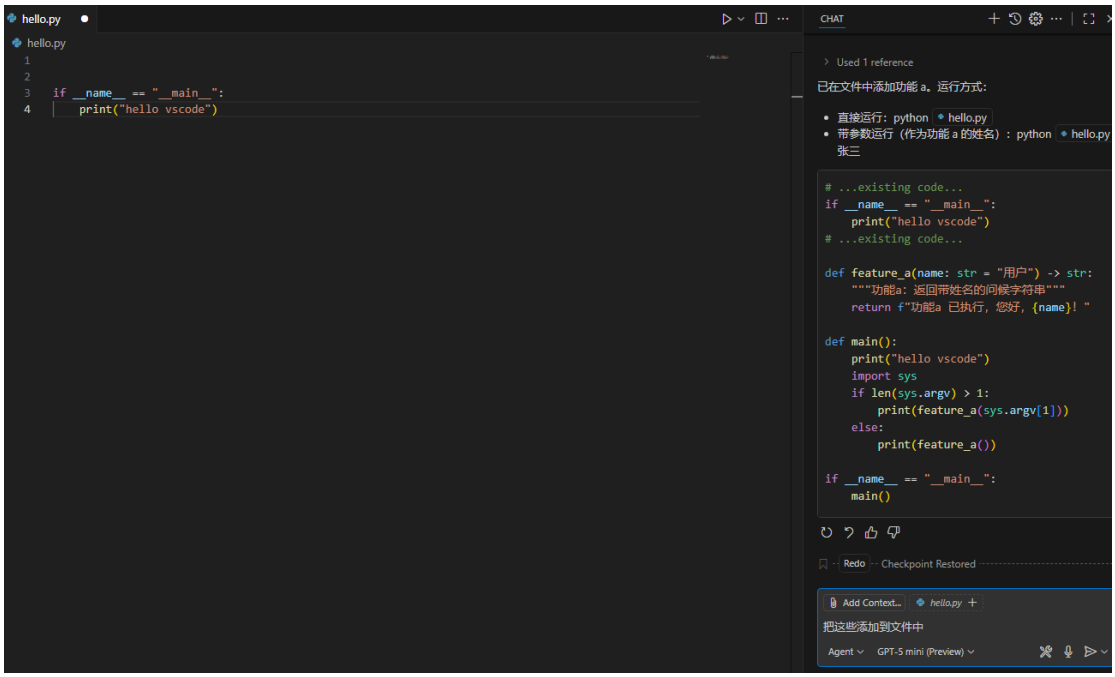


模型执行完成后，hello.py 文件中出现了新的内容。可以检测内容是否是需要的，然后选择 Keep/Undo。



每段对话上方都有一个 checkpoint，点击后可以将操作撤销，回到执行这个操作之前。


CHAT
def main():
print("hello vscode")
import sys
if len(sys.argv) > 1:
print(feature_a(sys.argv[1]))
else:
print(feature_a())
Do you want to undo your last edit?
This will remove your last request and undo the edits made to hello.py. Do you want to proceed?
Don't ask again
Yes
Cancel
Restore Checkpoint
Restores workspace and chat to this point
把这些添加到文件中
I'll update hello.py to add feature_a and a main runner, then run the script (with and without an argument) to verify the changes. Expected outcome: hello.py contains the new functions and the terminal shows the printed greetings.
hello.py +35 -1
I'll run the script twice: once without arguments, and once with the argument '张三' to verify output. I'll run two terminal commands in sequence.
1 file changed


hello.py
if __name__ == "__main__":
print("hello vscode")
CHAT
Used 1 reference
已在文件中添加功能 a。运行方式：
直接运行：python hello.py
带参数运行（作为功能 a 的姓名）：python hello.py 张三
...existing code...
if __name__ == "__main__":
print("hello vscode")
...existing code...
def feature_a(name: str = "用户") -> str:
"""功能a：返回带姓名的问候字符串"""
return f"功能a 已执行，您好，{name}！"
def main():
print("hello vscode")
import sys
if len(sys.argv) > 1:
print(feature_a(sys.argv[1]))
else:
print(feature_a())
if __name__ == "__main__":
main()
Redo
Checkpoint Restored
Add Context...
hello.py
把这些添加到文件中
Agent
GPT-5 mini (Preview)